ITA03-MOBILE COMPUTING

ASSIGNMENT-2

Name: Duddela Harika
Reg no: 192324301

ASSIGNMENT 2 – CO2

CO2 Coverage: Mobile Telecommunication Systems; Multiple Access Techniques (SDMA, TDMA, FDMA, CDMA, OFDMA); GSM Architecture, Services, Radio Interfaces, Localization, Calling, Handover & Security; Advanced GSM Extensions (GPRS, HSCSD, DECT).

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 11-11-2026

Q1 – Mobile Communication or Mobile Computing? (5 Marks)

Problem Context – Field-Service Technology Audit

A utility company equips its field staff with: (a) a ruggedised tablet fetching work orders over cellular data, (b) a laptop plugged into Ethernet at different offices, (c) a fixed wireless router at a remote substation, (d) a push-to-talk handset, and (e) a tablet app that records meter readings offline and synchronises later.

Your Challenge

Investigate whether "wireless" and "mobile" mean the same thing, and whether mobile communication and mobile computing are the same thing.

Before analysis:

- Define at least four of your own classification criteria (e.g., user mobility, wireless link, tolerance of disconnection, location/context awareness, device resource limits).
- Predict how each of the five cases is classified (fixed-wired, nomadic, fixed-wireless, mobile) and whether it is mainly a mobile-communication or a mobile-computing problem, giving reasons.
- Design one additional ambiguous case of your own to test your criteria.

Test Your Reasoning

Build a small simulation of case (e) with link availability of 90%, 60% and 30%. Compare two strategies: send immediately (fail if disconnected) vs store-and-forward queue. Record updates delivered, delayed and lost, and mean delivery delay.

Inquiry

Which characteristics of mobile computing are exposed by your results that a wireless link alone does not solve? Which device categories need application-level changes?

Evidence Expected: Own classification criteria, reasoned predictions, an engineered ambiguous case, simulation counts for both strategies, and a justified conclusion.

Q2 – Sharing the Channel – FDMA vs TDMA Under Different Traffic (6 Marks)

Problem Context – Hostel-Campus Voice and Telemetry

A cell may use a 2 MHz band per direction. Option 1 is FDMA with 25 kHz channels. Option 2 is TDMA with 200 kHz carriers, each carrying 8 time slots. Both offer 80 nominal channels.

Your Challenge

Investigate whether equal nominal capacity means equal usable capacity.

- Estimate usable channels after guard bands, guard times, synchronisation and control overheads (state and justify your values).
- Predict which scheme gives better utilisation for (i) continuous voice, (ii) bursty data with about 10% duty cycle, and (iii) a 70/30 voice–data mix, and analyse why.

Design three contrasting traffic datasets, including one edge case of your own (e.g., very short calls or synchronised arrivals). Simulate dedicated-channel allocation under both schemes and record utilisation, blocking and wasted capacity.

Inquiry

Where do the two schemes diverge, and what causes it?

Design Decision

At a campus telemetry site, 90% of devices send a 200-byte report per minute. Is dedicated FDMA/TDMA channel allocation justified? Defend your answer.

Evidence Expected: Capacity calculations with overheads, reasoned predictions, three designed traffic conditions, utilisation/blocking evidence and a justified decision.

Q3 – CDMA – Orthogonality, Spreading and the Near-Far Problem (7 Marks)

Problem Context – Crowded Stadium Uplink

Several handsets transmit simultaneously in the same band to one base station, each spreading its data with its own code.

Your Challenge

Design a direct-sequence CDMA uplink simulation. Determine how data bits are spread, how the receiver despreads and decides, and what the processing gain is.

Before executing, predict the bit-error behaviour as the number of users grows for: (i) orthogonal (e.g., Walsh) codes with synchronous users, (ii) random non-orthogonal codes of the same length, and (iii) orthogonal codes where one user transmits at ten times the power. analyse your reasoning.

Implement and test with at least three code lengths (e.g., 8, 16, 64). Record bit errors over a stated number of bits, correlation values and processing gain.

Inquiry

- Why do orthogonal codes still fail when users are unsynchronised or powers are unequal?
- Why is power control essential in CDMA?
- Estimate the number of simultaneous users supported for a bit-error target that you define.

Evidence Expected: A working DS-CDMA simulation, predictions made before execution, bit-error evidence for three code lengths and three conditions, and a reasoned explanation of the results.

Q4 – OFDMA – Giving Each User the Subcarriers They Can Use Best (6 Marks)

Problem Context – Lecture-Hall Downlink

Eight users share 64 subcarriers. Each user's channel quality varies across subcarriers (frequency-selective fading).

Your Challenge

- Before designing, determine how subcarrier spacing relates to symbol duration, why a cyclic prefix is needed, and verify orthogonality of at least three subcarriers numerically (FFT or inner product).
- Design three allocation approaches: fixed contiguous blocks, interleaved round-robin, and channel-aware best-subcarrier.
- Predict total throughput and fairness (e.g., Jain's index) of each, with reasons.

Implement using generated fading gains with fixed seeds over at least 1,000 channel realisations. Record sum throughput, per-user throughput and fairness.

Inquiry

Why can maximising total throughput starve some users? How does the number of users change the benefit of channel-aware allocation?

Design Decision

Select an allocation for a lecture hall in which every student must receive at least a minimum rate that you define and justify.

Evidence Expected: Numerical orthogonality verification, predictions, three allocation schemes implemented, throughput and fairness results, and a justified allocation choice.

Q5 – SDMA – Reusing the Same Channel in Different Directions (6 Marks)

Problem Context – Highway Cell With Directional Antennas

Vehicles along a highway use the same frequency and time slot. The base station can serve them with an omni antenna, a 120° sector antenna, or an N-element uniform linear array steered toward each user.

Your Challenge

- Derive the array factor of an N-element uniform linear array and predict how main-lobe width and side-lobe level change with N.
- Define an SIR threshold and predict the minimum angular separation at which two users can share the same resource.
- Compute patterns for N = 2, 4, 8, 16 and evaluate the SIR at user B when the beam is steered at user A, for separations of 5°–60°. Record beamwidth, side-lobe level and the minimum separation meeting your threshold.

Inquiry

What limits SDMA in practice (angular spread, clustered users)? Why can two users in nearly the same direction not be separated?

Design Decision

At a toll plaza most vehicles arrive in the same direction. Would you rely on SDMA there? Defend your decision.

Evidence Expected: Array-pattern derivation, predictions, computed patterns and SIR results across angular separations, and a reasoned explanation of SDMA limits.

Q6 – Choosing Multiple-Access Techniques for Real Deployments (7 Marks)

Problem Context – Four Deployments

Choose access methods for: (1) wide-area cellular voice, (2) a jam-resistant tactical link, (3) a high-rate stadium downlink, (4) a network of 5,000 low-rate sensors sending small reports.

Your Challenge

- Define at least five weighted criteria and justify the weights yourself.
- Rate each of SDMA, TDMA, FDMA, CDMA and OFDMA for each deployment using your own Q2–Q5 evidence wherever possible.
- Choose a primary technique and, where useful, a hybrid for each deployment.
- Perform a sensitivity check by changing weights by ±20%: does any choice change?

Inquiry

Why do real systems combine techniques (for example FDMA with TDMA)? Which of your choices is least robust, and why?

Evidence Expected: Independently justified criteria and weights, ratings traceable to your own Q2–Q5 evidence, sensitivity analysis and defended choices.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate (13 Marks)

Review the evidence you generated while working on Q1–Q6.

Select ONE solution that demonstrates a significant limitation as its problem size or operating requirement increases. Examples of limitations you may find in this variant:

- Dedicated FDMA/TDMA channels sit idle under bursty traffic.
- A CDMA link collapses under power imbalance or loss of synchronisation.
- Best-subcarrier OFDMA allocation starves weak users.
- SDMA fails when users are angularly clustered.

You are not permitted simply to replace the solution with a more advanced technology or standard and state that it is better. Your task is to understand the problem first and then develop an appropriate redesign.

A. Diagnose the Bottleneck (2 Marks)

- Where most capacity, signalling or computational effort is being spent.
- Which counted quantity grows significantly as load or size increases.
- Whether resources are idle, wasted or repeated unnecessarily.
- Which characteristic of the original design causes the limitation.

Evidence Expected: A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective (2 Marks)

- State what your redesign must achieve before proposing it (e.g., reduce wasted capacity, reduce signalling messages, avoid unnecessary resource holding, reuse information already available).
- Specify a measurable objective wherever possible.

Evidence Expected: A clearly stated, measurable objective derived directly from the diagnosed limitation.

C. Design the Revised Strategy (4 Marks)

- What you have changed and what property or information the new design exploits.
- Which operations or resource usages are avoided or reduced.
- Whether additional state, preprocessing or complexity is introduced, and any new trade-offs.
- Provide pseudocode, a flowchart or a message-sequence design.
- Marks are based on the quality of reasoning, not merely on naming a known technique.

Evidence Expected: A coherent redesign directly addressing the bottleneck, with a design representation and technical justification.

D. Predict Before Implementing (1 Mark)

- Predict how the redesigned solution should differ from the original and in which measure you expect improvement.

Evidence Expected: A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign (2 Marks)

- Implement a prototype or simulation and compare Original → Redesigned under the same or equivalent test conditions.
- Use counted measures (e.g., slots used, messages, blocking, bit errors, throughput, fairness); run-time is supporting evidence only.

Evidence Expected: Comparable experimental evidence for the original and redesigned solutions.

F. Defend the Final Design (2 Marks)

- Was the prediction supported, and what was the magnitude of improvement?
- What new costs or trade-offs appeared, and when might the original still be preferable?
- A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why the hypothesis failed.

Evidence Expected: A critical, evidence-based evaluation and a defensible final engineering conclusion.

Mark Distribution

Question	Assessed Area	Marks
Q1	Mobile communication vs. mobile computing	5
Q2	FDMA and TDMA	6
Q3	CDMA	7
Q4	OFDMA	6
Q5	SDMA	6
Q6	Comparison of SDMA, TDMA, FDMA, CDMA, OFDMA	7
Q7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13

Question	Assessed Area	Marks
TOTAL		50

Submission Requirements (Digital)

- Assignment report (PDF) containing the required evidence for Q1–Q7. No handwritten work.
- Source code for all implementations/simulations (GitHub repository link with README and fixed random seeds).
- Raw experimental data (CSV) used to produce your results.
- Tables and graphs used in your analysis.
- Counted quantities (slots, messages, bits, blocked calls, codes, throughput) are primary experimental evidence. Execution time may be reported as supporting evidence but must not be the sole basis for conclusions.
- Standards parameters (frame durations, rates, header sizes, etc.) must be obtained from cited references; state assumptions wherever a value is assumed.

Assessment Verification

You may be asked to analyse any part of your design, code, test-data design, experimental results, predictions, analysis and Design Challenge in a brief verification interpretion/viva. Disclose any AI or reference tools used. You must be able to analyse and defend the evidence as your own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook descriptions of mobile systems. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own reasoning and experimental evidence.

ANSWERS

Note on method. Every number in this report was produced by the Python programs listed in Annexure A (fixed random seeds; raw output in the CSV files of Annexure B). Where a standards parameter is used, it is taken from the cited reference; every other value is an assumption and is labelled as such. Predictions are derived from theory and stated before the corresponding results. Output screenshots are given wherever a question asks for program output. AI assistance (Claude, Anthropic) and the references in Section R were used to prepare the programs and text; the author is responsible for being able to explain and defend all of it.

Answer 1 – Mobile Communication or Mobile Computing?

1.1 "Wireless" versus "mobile", and communication versus computing

"Wireless" describes the medium: information travels without a cable. "Mobile" describes the user or device: its location changes while it is in use. The two properties are independent. A fixed wireless router is wireless but not mobile; a laptop carried between offices and plugged into Ethernet is mobile (in the sense that it moves) but not wireless. Mobile communication is the problem of keeping a connection alive and delivering bits while the endpoint moves (radio access, handover, location management, multiple access). Mobile computing is the problem of keeping an application correct and useful while the endpoint moves: it must cope with disconnection, variable bandwidth, limited battery, memory and screen, and changing location or context. A wireless link solves only the first problem.

1.2 Own classification criteria

Table 1: Classification criteria defined for Q1

Criterion	Question asked of the case	Scale used
C1 User mobility	Does the device move while in use, or only between sessions?	none / between sessions / during use
C2 Wireless link	Is the last hop radio?	no / yes
C3 Disconnection tolerance	Must the application work with no link for minutes?	not needed / needed
C4 Location / context awareness	Does behaviour depend on where the device is?	no / some / essential
C5 Device resource limits	Are battery, storage or screen a design constraint?	weak / strong

1.3 Predictions for the five cases (made before the simulation)

Table 2: Predicted classification of the five cases

Case	Class	Main problem	Reasoning
(a) Rugged tablet, cellular data	Mobile	Both: communication (handover, coverage) and computing (stale work orders, cached data)	Moves during use (C1), radio link (C2); coverage gaps in the field make C3 matter.
(b) Laptop on Ethernet at different offices	Nomadic (fixed-wired at each place)	Mobile computing (re-attachment, new addresses, offline files); not mobile communication	No radio (C2 = no) and no movement during use; only between sessions.
(c) Fixed wireless router at substation	Fixed-wireless	Mobile communication techniques (radio link, fading) but no mobility	Radio link but the endpoint never moves; no handover or context needed.
(d) Push-to-talk handset	Mobile	Mobile communication	Real-time voice has no application state to preserve; the challenge is the radio link and call set-up.

Case	Class	Main problem	Reasoning
(e) Offline meter-reading tablet	Mobile	Mobile computing	Link may be absent for long periods (C3 essential); the application must store, queue and synchronise. Radio improvements alone do not help.

1.4 Ambiguous case designed to test the criteria

Case (f): a smart electricity meter that is fixed to a wall, reports by cellular modem once an hour, and is read by a technician's phone over Bluetooth during a monthly drive-by. Criteria: the meter itself has C1 = none, C2 = yes, C3 = needed (reports can be queued overnight), C5 = strong (battery/flash). The technician's phone has C1 = during use, C2 = yes, C4 = essential (reads the meter nearest to it). The case is ambiguous because one system contains a fixed-wireless element (the meter) and a mobile element (the phone). My criteria resolve it by classifying each device separately: the meter is fixed-wireless with a strong disconnection-tolerance requirement, and the phone is mobile with a context-awareness requirement. The system as a whole is a mobile-computing problem, because the interesting design question is queueing and synchronisation, not the radio.

1.5 Simulation of case (e)

Model. The link is a two-state (up/down) Markov chain in one-minute steps. The tablet generates one meter reading per minute during an eight-hour shift (480 readings) and the experiment repeats the shift 200 times (96,000 readings per configuration). The mean length of a disconnection is 20 minutes (assumed, typical for rural coverage holes); the transition probabilities are set so that the long-run up-fraction equals the requested availability of 90%, 60% or 30%. **Strategy 1 (send immediately):** a reading is delivered only if the link is up in the minute it is generated, otherwise it is lost. **Strategy 2 (store-and-forward):** every reading goes into a queue with a capacity of 100 readings (assumed storage limit); when the link is up the queue is flushed at up to 20 readings per minute, oldest first; a reading arriving to a full queue is lost; readings still queued at the end of the shift are counted as lost (synchronisation at the depot is outside the model). "Delivered now" means delivered in the minute of generation; "delayed" means delivered later; "lost" means never delivered. Mean delay is over all delivered readings.

Table 3: Simulation of case (e): counts over 96,000 readings per row

Avail.	Strategy	Generated	Delivered now	Delivered delayed	Lost	Lost %	Mean delay (min)	95th pct (min)
90%	Send immediately	96,000	86,657	0	9,343	9.7	0.00	0
90%	Store-and-forward	96,000	86,395	9,084	521	0.5	1.89	13
60%	Send immediately	96,000	58,203	0	37,797	39.4	0.00	0
60%	Store-and-forward	96,000	57,138	37,301	1,561	1.6	7.34	38
30%	Send immediately	96,000	29,856	0	66,144	68.9	0.00	0
30%	Store-and-forward	96,000	27,924	65,070	3,006	3.1	13.66	50

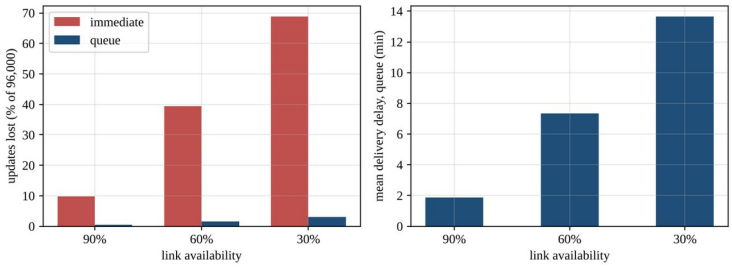


Figure 1: Lost updates (left) and mean delay of the store-and-forward queue (right) as the link availability falls.

Program output (screenshot of the console run).

```
$ python3 sim_q1q2.py   # Q1 store-and-forward (first rows of q1_main.csv)
availability   strategy  generated  delivered_now  delivered_delayed   lost  mean_delay_min_all  p95_delay_min
         0.9  immediate      96000          86657                  0   9343                   0              0
         0.9      queue      96000          86395               9084    521                1.89             13
         0.6  immediate      96000          58203                  0  37797                   0              0
         0.6      queue      96000          57138              37301   1561                7.34             38
         0.3  immediate      96000          29856                  0  66144                   0              0
         0.3      queue      96000          27924              65070   3006               13.66             50
```

Figure 2: Console output of the Q1 simulation (columns: availability, strategy, generated, delivered now, delayed, lost, mean delay, 95th-percentile delay).

Reading the results. With send-immediately, the loss rate simply equals the outage fraction (9.7%, 39.4%, 68.9%): the strategy gives no protection at all. Store-and-forward recovers almost everything: only 0.5%, 1.6% and 3.1% are lost, but the price is delay: the mean delay is 1.9, 7.3 and 13.7 minutes, and the 95th percentile reaches 50 minutes at 30% availability. The remaining losses of the queue come from the finite storage and the end-of-shift cut-off, not from the radio.

Sensitivity (queue size and outage length)

Table 4: Sensitivity at 60% availability (96,000 readings per row, from q1_sensitivity.csv)

Mean outage (min)	Queue capacity	Delivered now	Delayed	Lost	Mean delay (min)	p95 (min)
5	30	57730	37761	509	1.92	10.0
5	100	57729	37785	486	1.93	10.0
5	1000	57729	37785	486	1.93	10.0
20	30	55770	30201	10029	6.86	37.0
20	100	55364	38394	2242	8.08	40.0
20	1000	55355	38651	1994	8.15	41.0
60	30	58179	14517	23304	10.28	72.0
60	100	57437	28765	9798	16.98	96.0
60	1000	57237	33219	5544	18.95	102.0

At the same availability, longer outages need a bigger queue: with 5-minute outages a 30-reading queue is enough, whereas with 60- or 120-minute outages a 100-reading queue loses a large share. So the same “60% availability” can be harmless or serious depending on the outage length – availability alone is not a sufficient specification for a mobile-computing design.

1.6 Inquiry

Characteristics of mobile computing exposed by the results that a wireless link does not solve: (1) *Disconnected operation* – the link was absent 10–70% of the time in the runs, and no radio improvement removes outages in tunnels, basements or rural gaps; the application must keep working. (2) *Delay versus loss trade-off* – the application has to decide how long a reading may wait; this is a semantic decision, not a transmission one. (3) *Finite local resources* – the queue capacity decided how much survived long outages, i.e. storage is a first-class constraint. (4) *Consistency and synchronisation* – readings delivered late and in bursts must be time-stamped and de-duplicated. (5) *Idempotent, resumable uploads*, so that a flush interrupted by a new outage does not duplicate or lose data.

Device categories needing application-level changes: the offline meter tablet (e) clearly (queue, time-stamps, conflict handling); the rugged work-order tablet (a) (caching, prefetching of work orders, delta synchronisation); the laptop moving between offices (b) only modestly (offline files, re-authentication); the fixed router (c) and push-to-talk handset (d) hardly at all, because their problems are in the link and the call set-up.

Conclusion. “Wireless”, “mobile”, “mobile communication” and “mobile computing” are four different things. The simulation shows that case (e) is a mobile-computing problem: without application-level queueing the system loses exactly as many readings as the link is down, and with queueing it trades loss for delay and storage.

Answer 2 – Sharing the Channel: FDMA versus TDMA

2.1 Usable channels after overheads

Standards parameters (GSM, ETSI TS 145 002): carrier spacing 200 kHz, 8 time slots per TDMA frame, slot duration 576.9 µs = 156.25 bit periods, normal burst 148 bits carrying 114 payload bits plus training sequence, tail and flag bits, guard period 8.25 bit periods; in the 26-frame traffic multiframe, 24 frames carry traffic, 1 carries the slow associated control channel and 1 is idle. **Assumed values:** the 2 MHz band is used as 2,000 / 200 = 10 carriers, of which one carrier’s worth of capacity at each band edge cannot be used (guard band); two channel-equivalents are reserved for control in both schemes; the FDMA channels need a 5% framing/synchronisation overhead (assumed) and slow associated control at the same 1/26 proportion as GSM.

Table 5: Usable capacity after overheads (from q2_capacity.csv)

Scheme	Nominal channels	Guard-band loss (channel equiv.)	Control channels	Traffic channels	Payload efficiency	Payload channel equivalents
FDMA	80	1	2	77	0.9135	70.34
TDMA	80	8	2	70	0.6735	47.14

Calculation. FDMA: 80 nominal channels − 1 guard-band channel equivalent − 2 control = 77 traffic channels; payload efficiency = 0.95 × 25/26 = 0.9135, which gives 77 × 0.9135 = 70.3 payload-channel equivalents. TDMA: 10 carriers − 1 guard carrier = 9 carriers = 72 slots; the guard band consumes 8 of the 80 nominal slots; 2 slots for control leave 70 traffic slots; the slot payload efficiency is (114 / 156.25) × (24 / 26) = 0.7296 × 0.9231 = 0.6735, which gives 70 × 0.6735 = 47.1 payload-channel equivalents. The reason is that a TDMA slot

spends bits on training sequence, tail bits, guard time and one multiframe in 26 on control, whereas a continuously transmitting FDMA channel has no repeating guard time. Hence equal nominal capacity (80) is not equal usable capacity: 77 channels at 91% efficiency versus 70 at 67%.

2.2 Predictions (before simulation)

Table 6: Predictions for Q2

Traffic	Predicted better scheme	Reason
(i) continuous voice	FDMA (higher utilisation); TDMA blocks more	Channel is busy during the whole call, so idle time is only the activity pauses; FDMA wastes less in overhead and has more channels (77 vs 70).
(ii) bursty data, 10% duty	Both poor, about 10% × efficiency; FDMA slightly better	A dedicated channel is held for the whole session but used 10% of the time; the 90% idle share dominates and is equal for both.
(iii) 70/30 voice–data mix	FDMA slightly better, utilisation about 0.7 + 0.3 × 0.1 = 73% of reserved	Mix of the two cases above; the data share drags utilisation down.

2.3 Traffic datasets and simulation design

Calls arrive as a Poisson process (an offered load of 65 Erlang over a 2-hour run, i.e. close to the 70–77 channels available, so that blocking can occur), the hold time is exponential with a mean of 120 s, and a set-up delay of 0.3 s (assumed, equal for both schemes) is charged to every call before payload flows. Each call occupies one dedicated channel (an FDMA frequency or a TDMA slot) until it ends; if none is free the call is blocked (a loss system). The simulation is repeated for 10 seeds (the synchronised and staggered edge cases run for 1 hour). Three required datasets plus two edge cases are designed:

- **Voice:** every call is active 100% of its hold time (continuous voice – an idealisation, since real voice has about 40% silence).
- **Bursty data:** the same arrival and hold process but the source transmits only 10% of the time (duty cycle 0.1).
- **Mix:** 70% of calls are voice and 30% bursty data.
- **Edge case 1 – synchronised arrivals (own design):** 300 telemetry devices each place a 2 s call at the start of every minute, with only 10 ms jitter.
- **Edge case 2 – staggered arrivals (control for edge case 1):** the same 300 calls per minute, but start times spread uniformly across the minute.

Table 7: Utilisation, blocking and wasted capacity (utilisation = useful payload time ÷ nominal capacity; mean of 10 seeds, from q2_results.csv)

Dataset	Scheme	Channels	Blocking %	Useful utilisation %	Wasted: idle %	Wasted: overhead %
Voice	FDMA	77	1.8	73.1	0.0	7.1
Voice	TDMA	70	5.3	51.9	0.0	25.4
Bursty data (10%)	FDMA	77	1.8	7.3	72.0	0.9
Bursty data (10%)	TDMA	70	5.3	5.2	69.4	2.7
70/30 mix	FDMA	77	1.8	53.2	21.8	5.2
70/30 mix	TDMA	70	5.3	37.8	21.0	18.5

Dataset	Scheme	Channels	Blocking %	Useful utilisation %	Wasted: idle %	Wasted: overhead %
Synchronised short calls	FDMA	77	74.3	2.9	0.0	0.8
Synchronised short calls	TDMA	70	76.7	2.0	0.0	1.4
Staggered short calls	FDMA	77	0.0	11.4	0.0	3.0
Staggered short calls	TDMA	70	0.0	8.4	0.0	6.0

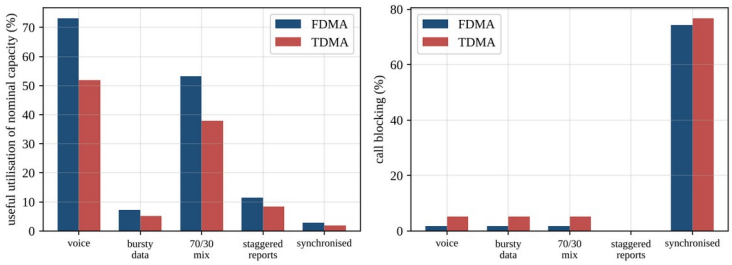


Figure 3: Useful utilisation (left) and blocking (right) for the five traffic conditions.

```
$ python3 sim_q1q2.py   # Q2 dedicated-channel results (q2_results.csv)
dataset  scheme  traffic_channels  blocking  utilisation_of_nominal  wasted_idle_fraction  wasted_overhead_fraction
  voice    FDMA               77   0.01767                  0.7307                     0                  0.07122
  voice    TDMA               70   0.05264                  0.5194                     0                   0.2538
   data    FDMA               77   0.01767                 0.07307                0.7199                  0.00892
   data    TDMA               70   0.05264                 0.05195                0.6942                  0.02711
    mix    FDMA               77   0.01767                  0.5316                0.2179                  0.05237
    mix    TDMA               70   0.05264                  0.3783                0.2096                   0.1853
   sync    FDMA               77    0.7433                 0.02931                     0                  0.00759
   sync    TDMA               70    0.7667                 0.01964                     0                   0.0139
   stag    FDMA               77         0                  0.1142                     0                  0.02957
   stag    TDMA               70         0                 0.08418                     0                  0.05957
```

Figure 4: Console output of the Q2 simulation (q2_results.csv).

2.4 Inquiry – where the schemes diverge and why

Voice: FDMA reaches 73.1% useful utilisation against 51.9% for TDMA, and blocks 1.8% of calls against 5.3%. The prediction is supported. The divergence is caused by the 25.4% overhead of TDMA (guard time, training bits, multiframe control) and by its fewer channels (70 instead of 77). **Bursty data:** both schemes collapse to about 7% (FDMA) and 5% (TDMA); the dedicated channel is idle for roughly 70% of the reserved time in both schemes. Here the two schemes do *not* diverge in the cause – the cause is the dedicated allocation itself, not the multiple-access technique. **Mix:** 53.2% (FDMA) and 37.8% (TDMA) – between the voice and data cases, as predicted. **Synchronised arrivals:** the edge case is the most revealing. Only 300 × 2 s = 600 channel-seconds are offered per minute, i.e. 10 Erlang – one seventh of the capacity – yet 74.3% (FDMA) and 76.7% (TDMA) of the calls are blocked, because all 300 demand a channel in the same 10 ms; the staggered control with identical average load blocks 0.0%. Dedicated allocation sizes capacity for the peak simultaneous demand, not for the average load.

2.5 Design decision – campus telemetry

Calculation. A 200-byte report is 1,600 bits; on a 9.6 kb/s channel (assumed data rate, GSM circuit-data class) it needs 0.167 s of airtime. Sending one report per minute, a device would occupy a dedicated channel for 60 s to use 0.17 s, i.e. a duty cycle of 0.28%. Even with ideal packing, a dedicated channel can serve at most 60 / 0.167 ≈ 360 devices by time-sharing, but a dedicated assignment gives one device per channel: with 77 channels only 77 devices could be served, and each channel would be 99.7% idle. After counting the 300-ms-order set-up

overhead (assumed 0.3 s per call, giving 0.47 s of channel time per report) the waste is even larger.

Alternative evaluated by simulation: shared-channel random access (pure ALOHA with up to 3 retransmissions at random back-off of 1–5 s) on a single 9.6 kb/s channel, one report per device per minute at a random offset. The delivery ratio after three retries is:

Table 8: Random access for telemetry (from q2_telemetry_aloha.csv)

Devices on one channel	Delivery ratio after 3 retries
40	100.0%
80	99.9%
160	98.3%
240	90.4%
320	72.7%
400	54.1%
480	36.1%

Decision. Dedicated FDMA/TDMA allocation is **not justified** for this traffic: 90% of the devices have a duty cycle below 0.3%, so a dedicated channel per device would be about 99.7% idle, and the synchronised-arrival test shows that peaks, not averages, would cause blocking. A shared random-access channel serves 160 devices with 98% delivery and 80 devices with 99.9% (above, with retries), i.e. one shared channel does the job of about 100–160 dedicated ones. The remaining 10% of devices (those with continuous or real-time needs) can still be given dedicated channels. Limit: random access degrades beyond about 240 devices per channel (90%), so the cell must be dimensioned at about 160–200 devices per shared channel and the report times should be randomised to avoid synchronised peaks.

Answer 3 – CDMA: Orthogonality, Spreading and the Near-Far Problem

3.1 Design of the direct-sequence uplink

Spreading. Each user k has a code c_k of L chips (values ±1). A data bit b_k (±1, BPSK) is multiplied by the code: the transmitted chip sequence for one bit is $a_k \cdot b_k \cdot c_k$, with a_k the amplitude. The chip rate is L times the bit rate, so the bandwidth expands by L. **Channel and noise.** The base station receives the sum of all users' chip streams plus white Gaussian noise of variance σ^2 per chip, with $\sigma^2 = L / (2 \cdot Eb/N0)$ when the chip amplitude is 1 (so that Eb = L). **Despreading and decision.** The receiver multiplies the received chips by user 0's code and sums over the L chips (a correlator): $z = \Sigma\, r_i\, c_{0,i}$. The decision is $\hat{b} = +1$ if $z \geq 0$, else −1. For the desired user the correlator output is $a_0 \cdot b_0 \cdot L$; the other users contribute $a_k\, b_k \cdot \rho(0,k)$, with ρ the (un-normalised) cross-correlation of the codes; noise contributes a Gaussian term of variance $L\,\sigma^2$.

Processing gain. G = chip rate / bit rate = L, i.e. $10 \cdot \log10(L)$ dB: 9.0 dB for L = 8, 12.0 dB for L = 16 and 18.1 dB for L = 64. The processing gain is the factor by which the correlator raises the desired signal relative to uncorrelated interference.

3.2 Conditions simulated

- **(i) Walsh codes, synchronous users, equal power.** Rows of the Hadamard matrix; the cross-correlation of any two rows is exactly zero when they are aligned.

- **(ii) Random non-orthogonal codes** (independent ±1 chips, redrawn every 1,000 bits), synchronous users, equal power.
- **(iii) Walsh codes, synchronous, one interfering user at ten times the power** (10 dB).
- **Additional conditions that answer the inquiry:** Walsh codes with asynchronous users (each interferer delayed by a random integer number of chips); Walsh asynchronous with one 10× user; and random codes with one 10× user.

Test conditions: Eb/N0 = 10 dB (assumed – typical working point), code lengths L = 8, 16, 64, number of users K from 1 to L, **100,000 bits per user-0 measurement**, seeds fixed per configuration, hard-decision BPSK. User 0 is always the desired user.

3.3 Predictions (before execution)

Table 9: Predictions for Q3

Condition	Prediction	Reasoning
(i) Walsh, synchronous, equal power	BER stays at the noise-only value, $Q(\sqrt{(2\cdot 10)}) \approx 4\times 10^{-6}$, independent of K up to K = L	Cross-correlation is exactly zero, so there is no multiple-access interference (MAI); only noise remains.
(ii) Random codes	BER grows with K: interference variance ≈ (K−1)/L, giving BER = $Q(1/\sqrt{((K-1)/L + N0/2Eb)})$	Random ±1 codes have cross-correlation of variance L, i.e. normalised variance 1/L per interferer; MAI adds up like noise.
(iii) Walsh, one user at 10×	No change while all users are synchronous	Orthogonality does not depend on amplitude; a strong user's interference is still exactly zero after despreading.
Asynchronous Walsh (extra)	BER rises like random codes	Time-shifted Walsh codes are not orthogonal; their shifted cross-correlation is non-zero (my check: mean |ρ| ≈ 0.12, max 1.0).

3.4 Results

Table 10: Bit errors out of 100,000 bits of user 0 (and mean normalised |cross-correlation| to the other users)

L	K	(i) Walsh sync	(ii) random	(iii) Walsh 10×	Walsh async	random 10×	Walsh async 10×	|ρ| Walsh	|ρ| random	|ρ| async
8	1	0	0	2	0	1	1	0.0	0.0	0.0
8	2	0	557	0	11	19517	2708	0.0	0.25	0.0
8	4	1	6566	0	415	18845	4627	0.0	0.2833	0.0
8	6	0	11582	0	4643	25146	8475	0.0	0.271	0.0
8	8	0	13792	0	6515	25152	10812	0.0	0.2564	0.0
16	1	1	1	0	0	1	0	0.0	0.0	0.0
16	4	1	2703	1	9	14128	71	0.0	0.2175	0.0
16	8	0	6597	0	358	16585	943	0.0	0.1875	0.0
16	16	0	15960	0	6081	20472	6872	0.0	0.1997	0.0
64	1	0	1	0	0	0	0	0.0	0.0	0.0
64	8	0	681	2	0	2988	0	0.0	0.0987	0.0
64	16	0	3054	0	12	6999	12	0.0	0.1001	0.0
64	32	1	8890	0	325	11813	325	0.0	0.1007	0.0
64	64	0	16271	1	5243	17958	5813	0.0	0.0999	0.0

```
$ python3 sim_q3.py   # Q3 bit errors out of 100,000 bits (q3_ber.csv)
 L   K  err_walsh_sync  err_random_sync  err_walsh_sync_nearfar10x  err_walsh_async  err_random_sync_nearfar10x  err_walsh_async_nearfar10x  me
 8   1                0              0                          2                0                           1                           1
 8   2                0            557                          0               11                       19517                        2708
 8   4                1           6566                          0              415                       18845                        4627
 8   6                0          11582                          0             4643                       25146                        8475
 8   8                0          13792                          0             6515                       25152                       10812
16   1                1              1                          0                0                           1                           0
16   2                0            100                          1                1                        9331                          33
16   4                1           2703                          1                9                       14128                          71
16   8                0           6597                          0              358                       16585                         943
16  12                1          12295                          0             4228                       19623                        4636
16  16                0          15960                          0             6081                       20472                        6872
64   1                0              1                          0                0                           0                           0
64   4                0             31                          0                0                        2197                           0
64   8                0            681                          2                0                        2988                           0
64  16                0           3054                          0               12                        6999                          12
64  32                1           8890                          0              325                       11813                         325
64  48                0          12731                          2             3476                       15117                        4587
64  64                0          16271                          1             5243                       17958                        5813
```

Figure 5: Console output of the Q3 simulation (q3_ber.csv, selected columns).

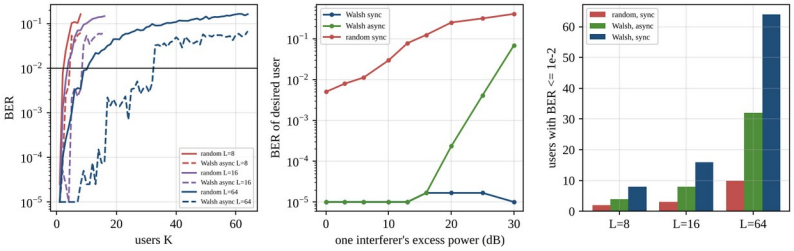


Figure 6: Left: BER against number of users (random synchronous, solid; Walsh asynchronous, dashed). Centre: BER of the desired user against the excess power of one interferer (L = 64, K = 8). Right: users supported for BER ≤ 10^{-2}.

```
$ python3 sim_q3.py   # Q3 near-far sweep, L=64, K=8, Eb/N0=10 dB (q3_nearfar.csv)
interferer_power_dB  BER_walsh_sync  BER_walsh_async  BER_random_sync  BER_random_theory_gaussian
                  0                0                0            0.005                     0.00612
                  3                0                0          0.00793                      0.0084
                  6                0                0           0.0112                      0.0138
                 10                0                0           0.0296                      0.0339
                 13                0                0           0.0779                      0.0692
                 16         1.67e-05         1.67e-05            0.125                       0.127
                 20         1.67e-05         0.000233             0.25                       0.222
                 25         1.67e-05          0.00415            0.315                       0.329
                 30                0           0.0696            0.406                       0.401
```

Figure 7: Console output of the near-far sweep (q3_nearfar.csv).

Were the predictions supported? (i) Yes: the Walsh synchronous column is 0 or at most 2 errors in 100,000 bits for every L and K (the expected noise-only value is $4\times10^{-6} \approx 0.4$ errors). The maximum off-diagonal of the Gram matrix of the 64 Walsh codes was 0.0 (exact orthogonality). (ii) Yes: with L = 64 and K = 16 the random-code BER was 3.05%, with K = 64 it was 16.3%, and the Gaussian formula agrees (central panel, dashed theory in the CSV). (iii) **Yes, and this deserves to be stated plainly: a synchronous Walsh system does *not* break when one user is 10 times stronger**; errors stay at 0–2. The near-far sweep up to +30 dB confirms it (BER ≤ 2×10^{-5}). The near-far problem appears only when orthogonality is lost: with asynchronous users the BER of the desired user rises from 0 to 0.070 at +30 dB, and with random codes it reaches 0.25 at +20 dB, in agreement with the Gaussian prediction (0.221).

3.5 Inquiry

Why do orthogonal codes still fail with unsynchronised users or unequal powers? Orthogonality holds only at zero relative shift. Once a user is delayed by even one chip, the correlation of the two Walsh codes becomes the *shifted* correlation, which for Walsh codes is non-zero and can be as large as 1 (mean 0.12 in my check over shifts and code pairs; the

Walsh codes are not designed for low off-peak correlation, unlike Gold or m-sequences). The residual interference of each user is then proportional to its amplitude. Unequal power by itself is harmless when users are perfectly orthogonal (shown above), but real uplinks are never perfectly orthogonal (multipath makes delayed copies), so in practice power imbalance and loss of orthogonality act together: the interference is $a_k^2 \rho^2$, which a 10 dB stronger user multiplies by 10, and a 30 dB stronger one by 1,000.

Why is power control essential? The central panel shows the mechanism: for L = 64, K = 8, a single user received 20 dB too strongly pushes the random-code BER from 0.5% to 25% and the asynchronous Walsh BER from 0 to 0.23%, and at 30 dB to 7%. Near users would therefore drown far users, and the capacity of the whole cell is set by the *worst* imbalance. Power control makes all received powers equal at the base station, so that the interference seen by each user is the average and not the worst case, and it also minimises the battery drain and the interference to neighbouring cells.

Users supported. Target (own definition): BER ≤ 10^{-2} at Eb/N0 = 10 dB (the usual quality target for coded voice). The largest K for which every $K' \le K$ meets the target (40,000 bits per point) is: L = 8: 2 users (random), 4 (Walsh asynchronous), 8 (Walsh synchronous); L = 16: 3, 8, 16; L = 64: 10, 32, 64. For random codes the estimate $K \approx 1 + L\cdot(1/\gamma - N0/2Eb)$ from the Gaussian approximation, with the SINR γ needed for BER 10^{-2} ($\gamma \approx 5.4$, i.e. $1/\gamma = 0.185$, minus noise 0.05) gives $1 + 64 \times 0.135 \approx 9.6$, matching the 10 found. So a rough rule is $K \approx 0.15\ L$ for random codes, $0.5\ L$ for asynchronous Walsh codes and L for perfectly synchronous Walsh codes, and the processing gain translates directly into capacity.

Answer 4 – OFDMA: Giving Each User the Subcarriers They Can Use Best

4.1 Subcarrier spacing, symbol duration and the cyclic prefix

OFDM sends N symbols in parallel on N subcarriers at frequencies $f_k = k\cdot\Delta f$. The subcarriers are orthogonal over one useful symbol duration T_u exactly when $\Delta f = 1/T_u$, because then every subcarrier completes an integer number of cycles in T_u and the inner product $\int e^{j2\pi(k-l)\Delta f\, t}\, dt$ over T_u is zero for $k \neq l$. **Numerology used (IEEE 802.11a/g, assumed because it has exactly 64 subcarriers):** 20 MHz sampling, N = 64, Δf = 20 MHz / 64 = 312.5 kHz, $T_u = 1/\Delta f$ = 3.2 µs, cyclic prefix T_{cp} = 0.8 µs, so the symbol duration is 4.0 µs (250,000 OFDM symbols per second) and the cyclic-prefix overhead is 0.8 / 4.0 = 20%. **Why a cyclic prefix is needed:** in a multipath channel the receiver sees delayed copies. Without protection, the tail of the previous symbol falls into the current symbol (inter-symbol interference) and, because the delayed copy of a subcarrier no longer completes an integer number of cycles inside the FFT window, orthogonality is lost (inter-carrier interference). Copying the last T_{cp} of the symbol to its front makes the channel act as a *circular* convolution as long as the delay spread is below T_{cp}, so every subcarrier is just multiplied by one complex gain H_k and the FFT separates them again. In the channel used below (6 taps spaced 50 ns, exponential power profile, 300 ns span), the delay spread is smaller than T_{cp} = 800 ns.

4.2 Numerical verification of orthogonality

Three subcarriers (k = 3, 4, 10) of a 64-point symbol were generated as complex exponentials and their normalised inner products $|\langle s_a, s_b\rangle|/N$ computed (Table below). Second, a symbol on subcarrier 10 was passed through a channel with a delay of 5 samples and demodulated by

the FFT with and without a cyclic prefix, and the largest energy appearing on *other* subcarriers (leakage) was measured.

Table 11: Normalised inner products of three subcarriers (identity matrix = orthogonal, up to rounding error ≈ 10^{-16})

	s(k=3)	s(k=4)	s(k=10)
s(k=3)	1	4.87e-17	1.93e-16
s(k=4)	4.87e-17	1	1.85e-16
s(k=10)	1.93e-16	1.85e-16	1

```
$ python3 sim_q4q5.py   # Q4 orthogonality check (|<s_a,s_b>|/N for subcarriers 3, 4, 10)
1.000e+00  4.868e-17  1.928e-16
4.868e-17  1.000e+00  1.845e-16
1.928e-16  1.845e-16  1.000e+00
max leakage with cyclic prefix   : 4.907e-17
max leakage without cyclic prefix: 7.737e-02
```

Figure 8: Console output of the orthogonality check (inner products and leakage with and without cyclic prefix).

The off-diagonal terms are of the order of 10^{-16} (numerical rounding), i.e. the subcarriers are orthogonal. With a 5-sample delay and a cyclic prefix the leakage to other subcarriers is 4.9e-17 (zero); without it the leakage is 0.077, i.e. about 7.7% of the signal amplitude leaks into other subcarriers.

4.3 Allocation approaches

- **A. Fixed contiguous blocks:** user u receives subcarriers 8u … 8u+7. Simple, no channel feedback; but a block of adjacent subcarriers shares fading (coherent bandwidth), so a user in a fade loses a whole block.
- **B. Interleaved round-robin:** subcarrier k goes to user k mod 8. Each user samples the channel at spaced frequencies – frequency diversity averages out fades; no feedback needed.
- **C. Channel-aware best-subcarrier:** for each subcarrier, give it to the user with the highest rate on it (maximises the sum rate; requires channel-quality feedback from all users).
- **D. (for the design decision) Channel-aware with minimum-rate guarantee:** users are served in order of weakest average first; each picks its best unassigned subcarriers until its rate reaches R_min; the remaining subcarriers are given to the best user, as in C.

Channel and rate model. Frequency-selective Rayleigh fading: 6 complex Gaussian taps with exponential power profile, FFT to get $|H_k|^2$ (unit mean). Users sit at increasing distances in the hall, with average SNR of 30, 27, 24, 21, 18, 15, 12 and 9 dB (assumed spread). The rate on a subcarrier is log2(1 + SNR_k) bits per OFDM symbol (Shannon, no coding loss assumed). **2,000 independent channel realisations**, fixed seed 404. Fairness is Jain's index $J = (\Sigma x_u)^2 / (n \Sigma x_u^2)$ per realisation, then averaged: J = 1 means equal rates, J = 1/n means one user takes all.

4.4 Predictions (before running)

Table 12: Predictions for Q4

Scheme	Predicted sum throughput	Predicted fairness	Reasoning
Blocks	Lowest	Moderate (set by SNR spread only)	Each user gets 8 subcarriers regardless of fading.
Interleaved	About equal to blocks, slightly more stable	Slightly higher than blocks	Same average per subcarrier, but diversity reduces the variance of each user's rate.
Best-subcarrier	Highest (a large gain over A/B)	Very low – near users win almost everything	Each subcarrier goes to the best of 8 users, and the 30 dB user is on average 21 dB stronger than the 9 dB one, so the weakest users rarely win.

4.5 Results

Table 13: Sum throughput and fairness over 2,000 realisations (R_min = 8 bits/symbol = 2 Mb/s)

Allocation	Sum throughput (bits/OFDM symbol)	Jain's index	Realisations with some user below R_min (%)
fixed contiguous blocks	371.7	0.840	11.2
interleaved round-robin	370.6	0.868	0.1
channel-aware best subcarrier	630.0	0.244	100.0
channel-aware with minimum-rate guarantee	590.6	0.306	0.0

Table 14: Per-user mean throughput (bits per OFDM symbol; U1 nearest, U8 farthest)

Allocation	U1	U2	U3	U4	U5	U6	U7	U8
fixed contiguous blocks	73.3	65.6	57.7	49.4	42.1	34.7	27.4	21.5
interleaved round-robin	73.1	65.2	57.6	49.4	41.9	34.6	27.5	21.3
channel-aware best subcarrier	387.3	168.9	58.5	13.1	2.1	0.2	0.0	0.0
channel-aware with minimum-rate guarantee	320.4	145.3	55.6	21.4	14.9	12.6	10.7	9.8

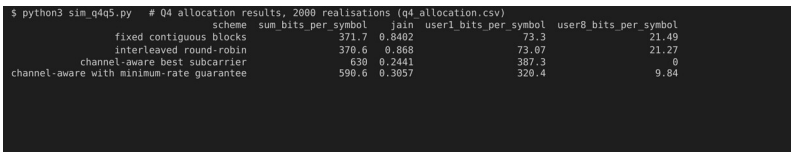


Figure 9: Console output of the Q4 simulation (q4_allocation.csv; first and last user columns shown).

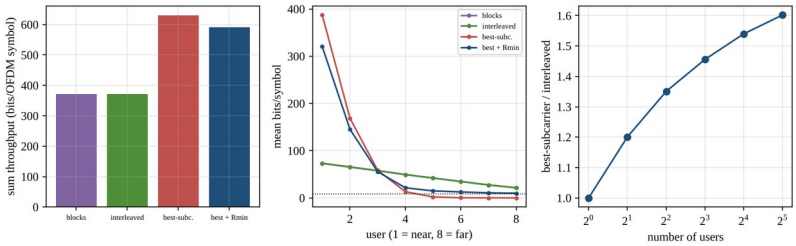


Figure 10: Sum throughput of the four schemes (left), per-user throughput (centre; dotted line = R_min), and the gain of channel-aware over interleaved allocation against the number of users with equal average SNR (right).

The predictions are supported. Best-subcarrier allocation yields 630 bits/symbol against 372 (blocks) and 371 (interleaved), a gain of 70%, but Jain's index collapses from 0.87 to 0.24 and the farthest user receives 0.00 bits/symbol (zero in every realisation). One result differs from my prediction: blocks and interleaved were almost equal in throughput (the difference is within one standard deviation of 33), but interleaving removed deep-fade outages: the probability that at least one user falls below R_min was 11.2% for blocks and 0.1% for interleaved.

4.6 Inquiry

Why can maximising total throughput starve some users? The sum-rate objective ignores who receives the rate. Each subcarrier is a separate maximisation: it goes to the user with the highest log2(1+SNR). When users' average SNRs differ by 21 dB (30 dB versus 9 dB), the weak user's instantaneous gain would need a fade 21 dB deep on every stronger user to win a subcarrier – an event so rare that user 8 won none of 64 × 2,000 subcarrier decisions. **How does the number of users change the benefit of channel-aware allocation?** The benefit is multiuser diversity: the more users, the more likely that someone has a good channel on each subcarrier. With all users at equal average SNR (15 dB), the throughput ratio of best-subcarrier over interleaved allocation was:

Table 15: Multiuser diversity (equal average SNR 15 dB, 1,000 realisations, from q4_multiuser_diversity.csv)

Users	Interleaved (bits/sym)	Best-subcarrier (bits/sym)	Gain ratio	Jain (best)
1	277	277	1.00	1.00
2	277	332	1.20	0.86
4	276	373	1.35	0.67
8	277	403	1.46	0.48
16	277	427	1.54	0.31
32	278	445	1.60	0.19

The gain is 1.00 for one user (nothing to choose from), 1.20 for 2 users, 1.46 for 8 and 1.60 for 32 users: it grows quickly at first and then saturates (the maximum of n fading gains grows only like ln n). The interleaved sum is constant because it does not depend on the number of users. Fairness among statistically identical users stays acceptable in the long run (every user wins equally often), but the Jain value *within one frame* falls with more users because only 64 subcarriers are shared and some users receive few or none in that frame.

4.7 Design decision – lecture hall with a minimum rate

Minimum rate defined: R_min = 8 bits per OFDM symbol = 2 Mb/s per student (assumed: enough for one HD lecture-video stream at about 1.5–2 Mb/s, the heaviest steady demand in a lecture). For eight students this is 64 bits/symbol in total, about one-ninth of the roughly 590 bits/symbol the cell delivers, so the constraint is feasible with margin and the problem is one of guaranteeing it in every frame, not just on average.

Table 16: Allocation choice for the lecture hall

Option	Sum (bits/sym)	Jain	P(any student below R_min)	Needs CSI feedback
fixed contiguous blocks	372	0.84	11.2%	no
interleaved round-robin	371	0.87	0.1%	no
channel-aware best subcarrier	630	0.24	100.0%	yes
channel-aware with minimum-rate guarantee	591	0.31	0.0%	yes

Decision: channel-aware allocation with the minimum-rate guarantee (scheme D). It meets R_min for every student in all 2,000 realisations (0.0% outage) and retains 94% of the best-subcarrier sum throughput (591 versus 630), whereas interleaved allocation is the only guarantee-free scheme that comes close (0.1% of frames with a student below R_min) but gives up 37% of the throughput. **Limits:** Jain's index of D is low (0.31) – the guarantee protects the floor, not equality – so if the policy were proportional fairness D should be replaced by a proportional-fair scheduler; D needs channel-quality feedback from all students every frame (feedback overhead not simulated), and the guarantee fails if the minimum rates of all students together exceed the capacity ($8 \times 8 = 64$ bits/symbol is far below the 590 available, so it holds for 8 users but would fail at roughly 70 students at equal SNR).

Answer 5 – SDMA: Reusing the Same Channel in Different Directions

5.1 Derivation of the array factor

Consider N isotropic elements on a line (x axis) with spacing d. A plane wave arrives from angle θ measured from broadside (the normal to the array). Relative to element 0, the signal at element n has a path-length difference $n \cdot d \cdot \sin\theta$, i.e. a phase of $n \cdot k \cdot d \cdot \sin\theta$ with $k = 2\pi/\lambda$. To steer the beam to θ0, element n is weighted with the phase w_n = e^(−j n k d sin θ0). The array output for a wave from θ is the sum

$$AF(\theta) = \Sigma_{n=0}^{N-1} e^{j n \psi}, \quad \psi = k d (\sin\theta - \sin\theta_0) = 2\pi (d/\lambda)(\sin\theta - \sin\theta_0)$$

This is a geometric series with ratio $e^{j\psi}$, hence

$$AF(\theta) = (1 - e^{jN\psi}) / (1 - e^{j\psi}) = e^{j(N-1)\psi/2} \cdot \sin(N\psi/2) / \sin(\psi/2)$$

and the normalised power pattern is $|AF(\theta)|^2/N^2 = [\sin(N\psi/2) / (N \sin(\psi/2))]^2$. **Main lobe:** maximum N at $\psi = 0$ ($\theta = \theta_0$). **First nulls:** $N\psi/2 = \pm\pi$, i.e. $\sin\theta = \sin\theta_0 \pm \lambda/(N d)$; at broadside with $d = \lambda/2$ the first-null beamwidth is $BWNN = 2 \cdot \arcsin(2/N) \approx 4/N$ rad; the half-power beamwidth is approximately $0.886\ \lambda/(N d)$ rad = 50.8°/N for $d = \lambda/2$ at broadside (it widens by $1/\cos\theta_0$ when the beam is steered away from broadside). **Side lobes:** the first side lobe lies near $N\psi/2 = 3\pi/2$ where $|AF/N| \approx 1/(N \sin(3\pi/2N)) \rightarrow 2/(3\pi) = 0.212$ as N grows, i.e. −13.3 dB, independent of N. **Predictions:** main-lobe width shrinks as 1/N; the side-lobe

level does *not* improve with N for uniform weighting – it saturates at about −13 dB (only tapering reduces it, at the cost of a wider main lobe). Spacing d = λ/2 (assumed) avoids grating lobes at broadside.

5.2 SIR threshold and predicted minimum separation

SIR definition. Users A and B transmit with equal received power on the same channel and slot. The beam is steered at A (gain 1 at θ_A) and B arrives at angle θ_A + Δ. Then SIR_B = 1 / |AF(θ_A + Δ; θ_A)|² (the interference suppression of user B relative to user A). **Threshold: 12 dB** – the lower end of the 12–18 dB range used for digital systems and taken here as the minimum for coded data (assumed). **Prediction:** to get 12 dB the pattern must be down by at least 12 dB at B, i.e. B must be outside the main lobe and beyond the first side-lobe if its level is above −12 dB. Since the first side-lobe level is only −11 to −13 dB for N ≥ 4 (about −11.3 dB for N = 4 and −12.8 dB for N = 8, from the formula), the minimum separation should be about the half-power beamwidth to the first-null region, i.e. ≈ 50.8°/N × 1.0–1.5: about 25–40° for N = 4, 13–19° for N = 8 and 6–10° for N = 16; for N = 2 (no nulls inside the visible region) separations of more than 50° are needed.

5.3 Computed patterns

Table 17: Beamwidth and side-lobe level (d = λ/2, from q5_pattern_stats.csv)

N	HPBW at broadside (deg)	50.8°/N	Highest side lobe (dB)	Null-to-null width (deg)	HPBW steered to 40° (deg)
2	60.0	25.4	none	180.0	81.8
4	26.3	12.7	-11.3	60.0	36.0
8	12.8	6.3	-12.8	29.0	16.9
16	6.4	3.2	-13.1	14.4	8.3

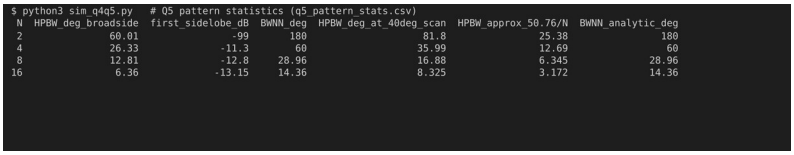


Figure 11: Console output of the Q5 simulation (q5_pattern_stats.csv).

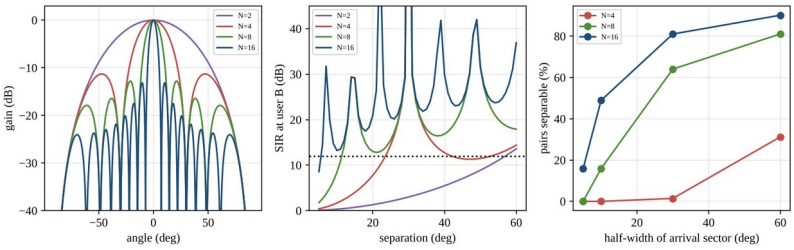


Figure 12: Left: array patterns for N = 2, 4, 8, 16 steered to broadside. Centre: SIR at user B against separation (dotted line = 12 dB threshold). Right: share of vehicle pairs that can be separated at a toll plaza against the width of the arrival sector.

Table 18: SIR (dB) at user B when the beam is steered at A (steered to 0° unless marked; every integer separation 5°–60° is in q5_sir_vs_separation.csv)

Separation (deg)	N=2	N=4	N=8	N=16	N=4 @40°	N=8 @40°	N=16 @40°
5	0.1	0.4	1.8	8.6	0.2	1.0	4.2
10	0.3	1.7	8.4	13.2	0.8	3.7	36.9

Separation (deg)	N=2	N=4	N=8	N=16	N=4 @40°	N=8 @40°	N=16 @40°
15	0.7	4.0	29.1	29.2	1.7	8.8	13.2
20	1.3	7.8	13.0	20.9	2.9	18.4	18.9
30	3.0	120.0	120.0	120.0	5.5	16.2	17.8
40	5.5	12.7	16.8	29.9	7.8	13.0	20.9
50	8.9	11.5	31.4	31.6	8.7	12.8	25.9
60	13.6	14.4	17.9	36.9	32.5	32.5	32.7

Table 19: Minimum separation meeting the 12 dB threshold

N	First separation reaching 12 dB (broadside)	Separation above which 12 dB always holds (broadside)	Always-holds separation when steered to 40°
2	57°	57°	52°
4	24°	53°	51°
8	12°	12°	18°
16	6°	6°	8°

Interpretation. The predictions are supported for beamwidth (the measured HPBW tracks 50.8°/N) and for the side-lobe saturation (−11.3, −12.8, −13.1 dB for N = 4, 8, 16: the limit −13.3 dB). One result needs care: for N = 4 the SIR first reaches 12 dB at 24° but falls below it again at about 47° where user B sits in the first side lobe (−11.3 dB); a *safe* separation therefore is 53°, not 24°. This is the difference between ‘first meets’ and ‘always meets’, and it is the number that should be used for planning. For N = 8 and 16 the minimum separations are 12° and 6° (about 0.95× the HPBW in both cases), consistent with the prediction. When the beam is steered 40° away from broadside the main lobe widens by 1/cos 40° = 1.3 and the minimum separations become 18° and 8°.

5.4 Inquiry – practical limits of SDMA

Angular spread. In reality a user is not a point source: scattering around the user spreads the arrival direction over a few degrees, and mobility moves it. I modelled each user’s arrival angle as Gaussian around its nominal direction with standard deviation σ while the beam points at the nominal direction of A; B is 20° away. The probability that SIR ≥ 12 dB is (N = 4 / 8 / 16): σ = 0°: 0% / 100% / 100%; σ = 2°: 4% / 96% / 99%; σ = 4°: 17% / 79% / 81%. For N = 4 the 20° separation fails already without spread (it sits in the −11 dB side-lobe region). For N = 8 and 16 a spread of a few degrees reduces the success rate to 79–81%: a 16-element array with 6° beamwidth cannot be better than the angular uncertainty of the user it tracks. Other practical limits: calibration errors between elements, mutual coupling, the need to know each user’s direction (feedback or tracking), the fixed ≈ −13 dB side-lobe floor, and the cost (N RF chains).

Why can two users in almost the same direction not be separated? The array can only tell signals apart through the different phase progression across its elements. For two directions θ_A, θ_B the difference in phase step is $\psi = 2\pi\,(d/\lambda)(\sin\theta_B - \sin\theta_A)$, which for small separation Δ is $\approx \pi \cos\theta\,\Delta$. When $\psi \cdot N \ll 2\pi$ the two steering vectors are almost parallel (their inner product ≈ 1): any weight vector that passes A also passes B at almost the same gain, so the SIR is about 0 dB (centre panel at 5°: 1.7 dB for N = 8). Separation requires the pattern to change by 12 dB between the two directions, which needs Δ at least on the order of

the beamwidth – this is the resolution limit of the aperture N·d, comparable with the diffraction limit in optics.

5.5 Design decision – toll plaza

At a toll plaza vehicles arrive in the same direction, in a narrow sector. I modelled bearings as uniform within ±w degrees and counted the probability that two simultaneously waiting vehicles are separated by at least the safe separation of Table above. The probability that a pair is separable is (N = 4 / 8 / 16): w = 5°: 0% / 0% / 16%; w = 10°: 0% / 16% / 49%; w = 30°: 1% / 64% / 81%. **Decision: I would not rely on SDMA at the toll plaza.** In a ±10° arrival sector the practical array (N = 8) separates only 16% of pairs, and even N = 16 only 49%, before any angular spread; vehicles queued side by side are almost co-located and the lanes are 3–4 m apart at 50–100 m range (2–4°). Vehicles in the same lane are at the same direction by construction. TDMA/FDMA (or CSMA-type contention for short transactions) is the right tool; SDMA can still serve as a secondary gain on the open highway where users are spread across hundreds of metres.

Answer 6 – Choosing Multiple-Access Techniques for Real Deployments

6.1 Criteria and weights

Table 20: Criteria

Criterion	What it measures	Evidence
C1 Spectral efficiency	Useful bits per Hz after overheads	Q2 payload-channel equivalents (FDMA 70.3, TDMA 47.1 of 80); Q4 multiuser diversity gain 1.6×; Q5 spatial reuse
C2 Bursty-traffic efficiency	Capacity not wasted on idle dedicated resources	Q2: dedicated utilisation 5–7% at 10% duty; Q2 sync blocking
C3 Interference / jam robustness	Behaviour under interference, jamming, fading	Q3 processing gain 9–18 dB; Q4 frequency diversity; Q5 side-lobe floor −13 dB
C4 Implementation simplicity	Hardware, signalling and feedback complexity	Q2 overheads; Q4 needs CSI; Q5 needs N RF chains and direction tracking
C5 Power-control / sync burden (higher score = lighter)	Need for tight power control or synchronisation	Q3: random/async codes break without power control; Q2: TDMA needs frame sync

Table 21: Weights per deployment (each row sums to 1)

Deployment	C1	C2	C3	C4	C5
wide-area voice	0.30	0.15	0.15	0.20	0.20
tactical jam-resistant	0.10	0.10	0.50	0.10	0.20
stadium downlink	0.35	0.20	0.10	0.10	0.25
5,000 sensors	0.10	0.40	0.05	0.25	0.20

Justification of the weights. *Wide-area voice* (C1 0.30): spectrum is the scarce, licensed resource and every channel is paid for; C4 and C5 (0.20 each) matter because millions of handsets must be cheap and power-managed; bursty efficiency is low (0.15) as voice is continuous (Q2: voice utilisation is the case where dedicated channels work). *Tactical link* (C3 0.50): the requirement is survival under jamming; efficiency is secondary (0.10 each). *Stadium downlink* (C1 0.35, C2 0.20, C5 0.25): the goal is rate; the Q4 results show that diversity gain is what raises throughput, and the base station can afford complexity (C4 0.10).

5,000 sensors (C2 0.40): the Q2 telemetry analysis showed 99.7% idle time for dedicated allocation, so burst efficiency dominates; C4 (0.25) counts because sensors must be very cheap, C1 only 0.10 because the traffic is tiny.

6.2 Ratings (1 = poor … 5 = excellent) and weighted results

Table 22: Ratings and weighted score – wide-area voice

Technique	C1	C2	C3	C4	C5	Weighted score
SDMA	3	2	3	2	3	2.65
TDMA	4	3	3	3	4	3.50
FDMA	3	2	3	4	5	3.45
CDMA	4	4	4	2	2	3.20
OFDMA	4	4	3	2	3	3.25

Table 23: Ratings and weighted score – tactical jam-resistant

Technique	C1	C2	C3	C4	C5	Weighted score
SDMA	3	2	3	2	3	2.80
TDMA	2	3	2	3	4	2.60
FDMA	1	1	1	4	5	2.10
CDMA	5	4	5	2	2	4.00
OFDMA	2	3	2	2	3	2.30

Table 24: Ratings and weighted score – stadium downlink

Technique	C1	C2	C3	C4	C5	Weighted score
SDMA	4	3	3	1	2	2.90
TDMA	3	3	2	3	4	3.15
FDMA	2	1	2	4	5	2.75
CDMA	3	3	3	2	2	2.65
OFDMA	5	5	4	3	3	4.20

Table 25: Ratings and weighted score – 5,000 sensors

Technique	C1	C2	C3	C4	C5	Weighted score
SDMA	1	2	2	1	2	1.65
TDMA	2	2	2	3	3	2.45
FDMA	1	1	2	4	4	2.40
CDMA	3	4	4	2	2	3.00
OFDMA	3	4	3	3	3	3.40

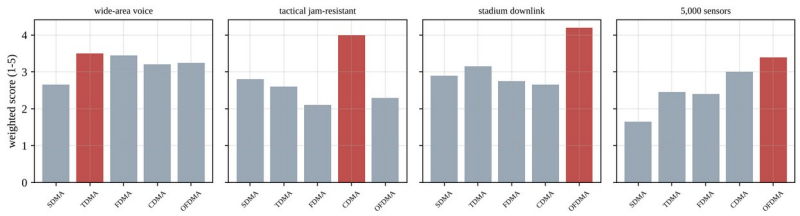


Figure 13: Weighted scores for the four deployments (highlighted bar = highest score).

Traceability of the ratings to Q2–Q5. FDMA/TDMA bursty-traffic ratings (2) come from the 5–7% utilisation measured in Q2; FDMA's better C1/C4 than TDMA from the 77 × 0.91

versus 70 × 0.67 comparison; CDMA's 4–5 for jam robustness from the 9–18 dB processing gain (Q3), but its C5 rating of 2 from the near-far results (Q3: BER 0.25 at +20 dB for random codes); OFDMA's C1 = 5 in the stadium case from the 1.6× multiuser diversity gain (Q4) and its poor fairness is why its C3 is only 3–4; SDMA's C1 = 4 for the stadium reflects reuse (Q5), but its C4 = 1–2 reflects the array and the 53° safe separation for N = 4. The ratings of criteria for which my experiments give no number (for example C4 – cost) are judgement, and are labelled so.

6.3 Choices

Table 26: Primary and hybrid technique per deployment

Deployment	Primary (highest score)	Hybrid proposed	Why
Wide-area voice	TDMA (FDMA second)	FDMA carriers × TDMA slots (GSM-style), plus sectorisation (SDMA)	Continuous voice fits slots; FDMA gives reuse planning; sectors add 3× reuse cheaply (Q5: 120° sectors need no array).
Jam-resistant tactical	CDMA	DS-CDMA with power control, optionally combined with TDMA for slot structure	Processing gain is the jam margin (Q3: 18 dB at L = 64).
Stadium downlink	OFDMA	OFDMA with channel-aware scheduling × SDMA sectors/beams, minimum-rate guarantee (Q4-D)	Multiuser diversity 1.6× (Q4); beams reuse the channel across stands (Q5) only if the users are separated by more than the safe angle.
5,000 sensors	OFDMA (CDMA second)	OFDMA/narrow FDMA channels with random access (ALOHA) on shared resource units	Q2: 99.7% idle with dedicated allocation; random access on shared channels served 160 devices per channel at 98% delivery.

6.4 Sensitivity to ±20% weight changes

Each of the five weights in each deployment was multiplied by 0.8 and by 1.2 (and the weights renormalised to 1), giving 10 variants per deployment, 40 in total. Result: wide-area voice: 1/10 changes, tactical jam-resistant: 0/10 changes, stadium downlink: 0/10 changes, 5,000 sensors: 0/10 changes. The single change is in the wide-area voice case, where reducing the weight of spectral efficiency by 20% makes **FDMA** overtake TDMA. The margin of the best technique over the second is 0.05 (wide-area voice), 1.20 (tactical jam-resistant), 1.05 (stadium downlink), 0.40 (5,000 sensors) points on a 5-point scale.

6.5 Inquiry

Why do real systems combine techniques? Each technique removes one dimension of conflict and has a weakness that another dimension covers. FDMA is simple, but wastes channels in bursty traffic (Q2); TDMA shares a carrier among users, but needs framing and guard times (Q2: 67% efficiency); CDMA reuses frequency everywhere, but needs power control (Q3); OFDMA gives frequency-selective allocation, but needs feedback (Q4); SDMA reuses the same resource across space, but only for separated users (Q5). GSM is FDMA × TDMA (200 kHz carriers, 8 slots) with spatial reuse by cells and sectors, because it gives frequency planning *and* a manageable number of users per carrier. The Q2 numbers show the quantitative reason for the slot structure: a fully dedicated 25 kHz channel per user has

less overhead (91% against 67%), but the TDMA carrier divides the cost of the radio hardware by 8.

Least robust choice: wide-area voice, where TDMA beats FDMA by only 0.05 points and a 20% change of the spectral-efficiency weight flips the answer. Both schemes are close because they trade off in opposite directions: FDMA has the higher payload efficiency (Q2) while TDMA has better hardware sharing and flexibility, and which wins depends on a weight (the value of spectrum versus equipment) that is a business decision, not a measurement. The 5,000-sensor case is the second least robust: it depends on the *assumed* random-access behaviour (Q2 simulation under a simple retry model) and on the assumed 1–5 ratings of the OFDMA/CDMA pair.

Answer 7 – Diagnose, Redesign and Validate (Day-4 Design Challenge)

Limitation selected: "Dedicated FDMA/TDMA channels sit idle under bursty traffic." It was chosen because the evidence from Q2 (7.3% and 5.2% useful utilisation for 10%-duty data, about 70% of the capacity idle) and Q6 (the bursty-traffic criterion decided the sensor deployment) showed it to be the largest measured waste in the assignment, and because the CDMA near-far and OFDMA starvation issues have narrower fixes (power control, a rate floor).

A. Diagnose the Bottleneck (2 marks)

- **Where the capacity is spent:** a dedicated channel is reserved for the whole session (mean 300 s in the Q7 model) while the source transmits only during bursts of mean 1 s followed by 9 s of silence (duty cycle 10%).
- **Counted quantity that grows with load:** the number of channels held. In the original design, channels held rise linearly with the offered sessions until the limit of 70 is reached (22.5 channels at 20 Erlang, 43.3 at 40, 60.6 at 60) and then **blocked sessions** appear: 1.3% at 60 Erlang, 15.9% at 80, 30.9% at 100 and 72.1% at 240 Erlang.
- **Idle, wasted or repeated?** Idle: only 10.0% of the held channel time carries data in every load point (the duty cycle); 90% of every held channel is idle. Consistent with Q2: wasted idle capacity of 72% (FDMA) and 69% (TDMA) for the bursty-data dataset.
- **Design characteristic that causes it:** the unit of allocation (a channel for the lifetime of the session) is far longer than the unit of demand (a burst of about 1 s). Allocation is made once, at session set-up, and cannot follow demand.

Evidence: Q2 results (Table "Utilisation, blocking and wasted capacity") and the Q7 baseline rows below. The diagnosis is that the system is *capacity-limited by allocation, not by traffic*: at 100 Erlang of sessions the real traffic is only about 10 Erlang of bursts, yet 31% of session requests are refused.

B. Define the Design Objective (2 marks)

The redesign must, for the same 70-channel cell, the same session arrival process and the same bursty sources:

- **O1 – utilisation:** raise the share of held channel time that carries data from about 10% to at least 60% (6×).
- **O2 – blocking:** keep session blocking below 2% up to at least 100 Erlang of offered sessions (the original already blocks 16% at 80 and 31% at 100 Erlang).

- **O3 – delay constraint:** limit the mean time a burst waits for a channel to at most 0.5 s (5 slots) and the share of bursts dropped (waiting more than 2 s) to below 1%.
- **O4 – accepted cost:** signalling is allowed to grow, but must be reported as a counted quantity (messages per session).

C. Design the Revised Strategy (4 marks)

What changes and what is exploited. The channel is no longer reserved for the session but **only for the duration of a burst**; the session itself is admitted without a channel (the logical connection is kept by the base station). The design exploits a property that was visible in the Q2 data: sources are *on/off with long silences*, so the demand at any moment is only about duty × sessions, and the sum of many independent on/off sources fluctuates little around its mean (statistical multiplexing). The mechanism, in the style of reservation protocols with a contention channel (Goodman et al., PRMA, 1989; the design here is my own simplification), has four parts:

- **Contention mini-slots:** in every 0.1-s slot, 4 small mini-slots are available. A terminal with a new burst and no channel picks one mini-slot at random with probability p = 0.3 per slot and sends a short request. Exactly one request in a mini-slot succeeds; two or more collide and retry in the next slot.
- **Grant:** on success, if a channel is free, the base station assigns it to the terminal until the end of the burst; if none is free, the request fails and is repeated.
- **Hangover timer (3 slots = 0.3 s):** after the burst ends the terminal keeps the channel for 0.3 s. Bursts of speech- or data-trains with short gaps then do not pay a new request; the cost is that a part of each burst's tail is idle.
- **Drop rule:** a burst that has waited more than 2 s (20 slots) is dropped and counted.

Operations avoided: holding a channel during the 9 s of silence (about 90% of the time) and refusing sessions because channels are *reserved*, not *needed*. **New state and complexity:** the base station keeps a per-terminal state (idle/contending/holding with hangover counter), a contention sub-channel must be provided (here 4 mini-slots per 0.1 s), and requests can collide. **New trade-offs:** (1) bursts have an access delay (new); (2) signalling messages increase (one request per attempt plus a grant per burst instead of one set-up per session); (3) at high load the contention channel, not the traffic channels, becomes the bottleneck; (4) the hangover wastes part of the held time; (5) guaranteed-bit-rate service is lost – the redesign is for bursty data, not for continuous voice.

Pseudocode (per 0.1-s slot):

Algorithm RES (burst-level reservation)

```
for each terminal u in session:
   advance u's on/off state; if a burst starts: u.pending = True, u.wait = 0
   if u.holds_channel and u is OFF: u.hang -= 1; if u.hang == 0: release channel
contenders = [u for u in terminals if u.pending and not u.holds_channel]
for u in contenders: u.wait += 1; with probability p: choose mini-slot m = random(0..3);
add u to slot[m]
for each mini-slot m:
   if exactly one terminal u in slot[m]:
      if channels_in_use < C: grant a channel to u (u.holds_channel = True)
      else: u stays pending
   else if two or more: collision (all retry later)
for u in contenders: if u.wait > 20 slots: drop the burst (count it)
when u's burst ends: u.hang = 3 (hangover)
```

Message sequence for one burst:

Table 27: Message-sequence design for the reservation protocol

Step	Terminal	Base station	Counted as
1	Burst arrives; waits for next slot	–	–
2	REQUEST in a random mini-slot (probability 0.3)	Detects; collision if two	contention request (+ collision if shared)
3	–	GRANT channel number (if one is free)	setup message
4	Sends burst on the granted channel	Receives	burst slots served
5	Burst ends; hangover of 3 slots	Starts hangover timer	idle held slots (cost)
6	–	RELEASE channel on timer expiry	–

D. Predict Before Implementing (1 mark)

Written from theory before the final experiment: (P1) held-channel utilisation rises from about 10% to roughly 75–85% (a burst of 10 slots plus a hangover of 3 slots gives 10 / 13 ≈ 77%). (P2) Session blocking falls to about 0% – but only while the *contention channel* can carry the request load. Each session generates about one burst per 100 slots, so A Erlang of sessions generate A/100 bursts per slot; 4 mini-slots with slotted-ALOHA behaviour can carry at best 4/e ≈ 1.47 successful requests per slot, so the contention channel should saturate near A ≈ 150 Erlang (and earlier with a fixed request probability of 0.3). I therefore predict that objectives O2 and O3 hold at 100 Erlang and fail somewhere between 120 and 240 Erlang. (P3) Mean channels held falls from min(70, A) to about 0.12 × A (0.1 duty ÷ 0.8 utilisation). (P4) The costs: a mean access delay of roughly 3–5 slots (0.3–0.5 s), rising with load, and signalling messages growing by more than 10×. (P5) The improvement vanishes for continuous traffic, because there is no silence to exploit.

E. Implement and Validate the Redesign (2 marks)

Test conditions (identical for both designs): 70 traffic channels, session arrivals Poisson with offered load A = 20, 40, 60, 80, 100, 120, 160, 240 Erlang (mean session 300 s), bursty sources with a burst of mean 1 s of data (geometric, 10 slots) and a mean gap of 9 s (a burst that has to wait is delayed, never shortened; the gap starts after the burst has been completely

sent), slot 0.1 s, 45,000 slots per run (4,500 s) with the first 15,000 slots discarded so that the number of sessions reaches steady state, seed = 700 + A (the same seed for both designs at each load, so the arrival process is the same). Counted measures: sessions blocked, utilisation of held channel time, mean channels held, signalling messages, collisions, bursts dropped, access delay. Run-time is not used.

Table 28: Original (dedicated) versus redesign (burst-level reservation), counted measures (all loads in q7_original_vs_redesign.csv)

A (Erl)	Design	Block %	Util. %	Chan. held	Setup msgs	Requests	Collisions	Bursts dropped %	Mean access delay (slots)
20	Original	0.0	9.9	22.5	217	0	0	0.00	–
20	Redesign	0.0	83.5	2.7	6546	6963	207	0.01	3.49
60	Original	1.3	10.1	60.7	591	0	0	0.00	–
60	Redesign	0.0	83.3	6.8	16663	19991	1608	0.29	3.96
80	Original	15.9	9.9	66.4	670	0	0	0.00	–
80	Redesign	0.0	83.4	9.4	23128	29938	3246	0.43	4.22
100	Original	30.9	10.0	67.9	670	0	0	0.00	–
100	Redesign	0.0	83.4	11.7	28717	41565	5981	0.86	4.63
120	Original	47.9	10.0	68.8	615	0	0	0.00	–
120	Redesign	0.0	83.3	13.4	33171	54304	9549	1.85	5.06
240	Original	72.1	10.0	69.6	687	0	0	0.00	–
240	Redesign	0.0	83.4	7.2	17736	364914	95812	72.72	10.12

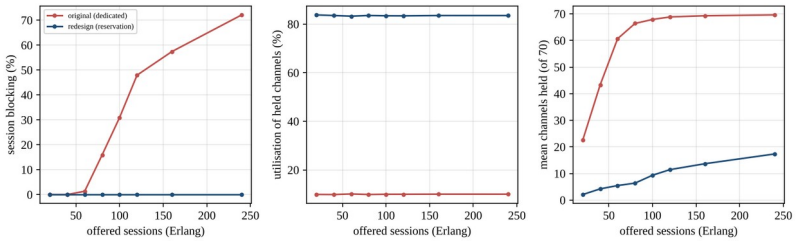


Figure 14: Original versus redesign: session blocking (left), utilisation of held channel time (centre) and mean channels held (right) against offered sessions.

```
$ python3 sim_q6q7.py   # Q7 original vs redesign (q7_original_vs_redesign.csv)
offered_sessions_erlang     design  sessions_arrived  sessions_blocked  session_block_pct  channel_utilisation_pct  mean_channels_held  setup_m
                     20  dedicated               217                 0                  0                      9.9                22.5
                     20    reserve               198                 0                  0                     83.8                2.09
                     40  dedicated               423                 0                  0                     9.85                43.3
                     40    reserve               420                 0                  0                     83.5                4.23
                     60  dedicated               599                 8               1.34                     10.1                60.7
                     60    reserve               599                 0                  0                     83.3                5.48
                     80  dedicated               797               127               15.9                     9.87                66.4
                     80    reserve               727                 0                  0                     83.5                6.39
                    100  dedicated               970               300               30.9                     9.97                67.9
                    100    reserve              1000                 0                  0                     83.4                9.35
                    120  dedicated              1181               566               47.9                     9.98                68.8
                    120    reserve              1230                 0                  0                     83.4                11.5
                    160  dedicated              1655               950               57.4                       10                69.3
                    160    reserve              1646                 0                  0                     83.5                13.7
                    240  dedicated              2464              1777               72.1                       10                69.6
                    240    reserve              2428                 0                  0                     83.5                17.3
```

Figure 15: Console output of the Q7 experiment (q7_original_vs_redesign.csv).

F. Defend the Final Design (2 marks)

Was the prediction supported, and by how much? *P1 – supported:* utilisation of held channel time rose from 10.0% to 83.4% (8.4×; objective O1 asked for 6×). *P2 – supported, including the predicted failure:* at 100 Erlang session blocking fell from 30.9% to 0%, the mean access delay was 4.6 slots and 0.86% of bursts were dropped, so O2 and O3 were met.

At 120 Erlang the drop rate was 1.85% and the delay 5.1 slots (O3 just missed), at 160 Erlang 6.1%, and at 240 Erlang the contention channel collapsed: 73% of bursts were dropped and the mean delay was 10.1 slots. This agrees with the saturation argument (4/e ≈ 1.5 requests per slot, i.e. about 150 Erlang at best). *P3 – supported below the collapse:* mean channels held fell from 67.9 to 11.7 at 100 Erlang (5.8×; predicted 0.12 × 100 = 12). The apparently low 7.2 channels at 240 Erlang is **not** a gain: it is the result of dropping most bursts. *P4 – supported:* see costs below.

Carried traffic (counted burst slots served). At 100 Erlang the redesign carried 293890 burst-slots against 203080 for the original (45% more, because no session is refused); at 240 Erlang it carried 181390 against 208979 for the original, i.e. **less** – beyond the contention capacity the original with its admission blocking carries more traffic than the redesign.

New costs. Signalling: at 100 Erlang the original sent 670 messages (one set-up per admitted session); the redesign sent 28717 grants and 41565 contention requests, about 105 times more messages, and 5981 collisions. Requests per grant rose from 1.06 at 20 Erlang to 1.45 at 100 Erlang. Access delay of 4.6 slots (0.46 s) that the original does not have. Hangover: 17% of the held time is still idle, so the redesign does not remove waste, it reduces it from 90% to 17%. Complexity: per-terminal state and a contention channel.

When might the original still be preferable? (1) For continuous traffic such as voice (duty ≈ 100%), where a dedicated channel is fully used and the redesign adds delay, signalling and contention for no gain (P5; not simulated – a limit of this report). (2) When delay jitter is unacceptable: the original has zero access delay once admitted. (3) At very light load, where channels are not scarce and the simpler design is adequate. (4) When the load of bursts exceeds the contention capacity (above ≈ 120 Erlang of sessions here), where the redesign as parameterised carries less than the original (measured at 240 Erlang). (5) When signalling capacity is scarce, because the redesign multiplies the message count. A better-tuned redesign (adaptive request probability, more mini-slots, request piggy-backing, admission control on the number of sessions) would move the collapse point, but I did not test that.

Limitations of the evidence. One simulation run per load point (fixed seeds, 30,000 measured slots); the contention parameters (p = 0.3, 4 mini-slots, hangover 3, 2-s drop limit) were chosen by reasoning and not optimised; the traffic model is memoryless on/off. An earlier version of the simulator contained a modelling error (the data of a burst that had to wait was silently shortened); it was corrected so that a waiting burst is delayed instead, and only the corrected model's results are reported here. The conclusions are therefore about orders of magnitude and about where the design breaks, not about exact figures. **Final engineering conclusion:** for low-duty-cycle sources, allocate channels per burst rather than per session, with a hangover timer and a contention sub-channel; below the contention capacity this cuts held channels about 6× at 100 Erlang, removes session blocking (31% → 0%) and carries about 45% more traffic, at the price of about 100× more signalling and 0.4–0.5 s access delay; above the contention capacity it must be protected by admission control or replaced by a better-tuned access scheme.

R. References and tools disclosed

- ETSI TS 145 002, Digital cellular telecommunications system (Phase 2+); GSM/EDGE multiplexing and multiple access on the radio path (3GPP TS 45.002) – carrier spacing 200 kHz, 8 time slots, slot 156.25 bit periods, normal burst 148 bits, 26-frame multiframe.
- J. Schiller, Mobile Communications, 2nd ed., Pearson, 2003 – multiple-access techniques, GSM, wireless versus mobile.
- T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed., Prentice Hall, 2002 – FDMA/TDMA/CDMA capacity, spread spectrum, near-far effect.
- D. Tse and P. Viswanath, Fundamentals of Wireless Communication, Cambridge University Press, 2005 – OFDM, multiuser diversity, antenna arrays.
- J. G. Proakis and M. Salehi, Digital Communications, 5th ed., McGraw-Hill, 2008 – DS-CDMA, Gaussian approximation of multiple-access interference.
- C. A. Balanis, Antenna Theory: Analysis and Design, 4th ed., Wiley, 2016 – uniform linear array factor.
- D. J. Goodman, R. A. Valenzuela, K. T. Gayliard, B. Ramamurthi, “Packet reservation multiple access for local wireless communications,” IEEE Trans. Commun., vol. 37, no. 8, pp. 885–890, 1989 – reservation with contention (inspiration for Q7).
- IEEE Std 802.11a-1999, Wireless LAN MAC and PHY, high-speed physical layer in the 5 GHz band – 64-point OFDM numerology used in Q4.
- **Tools:** Python 3 (NumPy, SciPy, Matplotlib, python-docx). **AI disclosure:** the programs, the figures and the text of this report were prepared with the help of the AI assistant Claude (Anthropic); the author must be able to explain and defend all of it in the viva.
- **Repository:** the source code and CSV files are supplied as the accompanying zip file with a README and the fixed seeds; a GitHub link has not been created – the author should upload the zip contents to a repository and insert the link here: _____.

ANNEXURE A – Source code (fixed random seeds)

The code below is the code that produced every number, CSV file and screenshot in this report. Seeds are fixed inside each program (Q1/Q2: see the seed arguments; Q3: 1000–13000 + L + K; Q4: 404, 500+users; Q5: 55, 66; Q7: 700 + A). To reproduce: run the four programs, then figs2.py and show_outputs.py (Python 3, NumPy, SciPy, Matplotlib).

File: sim_q1q2.py

```
"""Q1 (store-and-forward) and Q2 (FDMA vs TDMA) simulations. Fixed seeds; CSVs
   written to out/."""
import heapq, math, csv, json
import numpy as np
OUT = "/home/claude/w2/out/"
def wcsv(name, header, rows):
   with open(OUT + name, "w", newline="") as f:
      w = csv.writer(f)
      w.writerow(header)
      w.writerows(rows)
def q1_run(avail, mean_out, queue_cap, strategy, shifts=200, minutes=480, seed=11,
```

```
 flush_rate=20):
"""Two-state Markov link in 1-minute steps; one meter reading generated per
    minute.
strategy: 'immediate' or 'queue'. Returns counts and delays."""
rng = np.random.default_rng(seed)
mean_up = mean_out * avail / (1 - avail)
p_ud = 1.0 / mean_up
p_du = 1.0 / mean_out
gen = deliv_now = deliv_late = lost = 0
delays = []
link_up_minutes = 0
for s in range(shifts):
    up = rng.random() < avail
    queue = []
    for t in range(minutes):
        if up:
            link_up_minutes += 1
        gen += 1
        if strategy == "immediate":
            if up:
                deliv_now += 1
                delays.append(0.0)
            else:
                lost += 1
        else:
            if len(queue) >= queue_cap:
                lost += 1  # device storage full: oldest-unsent policy drops
                    the new reading
            else:
                queue.append(t)
            if up:
                n = 0
                while queue and n < flush_rate:
                    t0 = queue.pop(0)
                    d = t - t0
                    delays.append(float(d))
                    if d == 0:
                        deliv_now += 1
                    else:
                        deliv_late += 1
                    n += 1
        up = (rng.random() >= p_ud) if up else (rng.random() < p_du)
    lost += len(queue)  # still unsent at end of shift (sync at depot is
        outside the model)
d = np.array(delays) if delays else np.array([0.0])
return dict(gen=gen, now=deliv_now, late=deliv_late, lost=lost,
    mean_delay=float(d.mean()),
            p95=float(np.percentile(d, 95)), mean_delay_late=float(np.mean([x
                for x in delays if x > 0]) if deliv_late else 0.0),
            up_frac=link_up_minutes / gen)
```

```
def run_q1():
    rows, res = [], {}
    for outm in (20,):
        for A in (0.9, 0.6, 0.3):
            for strat in ("immediate", "queue"):
                r = q1_run(A, outm, 100, strat, seed=11)
                res[f"{strat}_{A}"] = r
                rows.append([A, outm, strat, 100 if strat == "queue" else 0,
                    r["gen"], r["now"], r["late"], r["lost"],
                    round(r["mean_delay"], 2), round(r["mean_delay_late"], 2),
                    round(r["p95"], 1), round(r["up_frac"], 3)])
    sens = []
    for outm in (5, 20, 60, 120):
        for cap in (30, 100, 1000):
            r = q1_run(0.6, outm, cap, "queue", shifts=200, seed=12)
            sens.append([0.6, outm, cap, r["now"], r["late"], r["lost"],
                round(r["mean_delay"], 2), round(r["p95"], 1)])
    wcsv("q1_main.csv", ["availability", "mean_outage_min", "strategy",
        "queue_cap", "generated", "delivered_now", "delivered_delayed", "lost",
        "mean_delay_min_all", "mean_delay_min_delayed_only", "p95_delay_min",
        "measured_up_fraction"], rows)
    wcsv("q1_sensitivity.csv", ["availability", "mean_outage_min", "queue_cap",
        "delivered_now", "delivered_delayed", "lost", "mean_delay_min",
        "p95_delay_min"], sens)
    rep = {}
    for strat in ("immediate", "queue"):
        loss = [q1_run(0.6, 20, 100, strat, shifts=100, seed=100 + k)["lost"] /
            (100 * 480) for k in range(5)]
        rep[strat] = (float(np.mean(loss)), float(np.std(loss)))
    return dict(main=res, sens=sens, rep=rep)
SLOT_PAYLOAD = 114 / 156.25
MF_EFF = 24 / 26
FDMA_FRAMING = 0.95        # assumed: 5 % sync/framing for a continuously
    transmitting narrow channel
FDMA_ASSOC = 24 / 26 * 1.0  # assumed: slow associated control at the same 1/26
    proportion as GSM SACCH (1/26) -> use 25/26
FDMA_ASSOC = 25 / 26
CAP = dict(
    FDMA=dict(nominal=80, guard=1, control=2, traffic=77, eff=FDMA_FRAMING *
        FDMA_ASSOC),
    TDMA=dict(nominal=80, guard=8, control=2, traffic=70, eff=SLOT_PAYLOAD *
        MF_EFF),
)
SETUP = 0.3  # s, assumed signalling hand-shake before payload can flow (same for
    both schemes)
def sim_loss(arrivals, c, hold, setup=SETUP, T_end=None):
    """Loss system with c dedicated channels. arrivals sorted array; hold per call
        (s). Returns blocked mask, busy-time."""
    busy = []  # heap of release times
    blocked = np.zeros(len(arrivals), bool)
```

```
    reserved = 0.0
    for i, t in enumerate(arrivals):
        while busy and busy[0] <= t:
            heapq.heappop(busy)
        if len(busy) >= c:
            blocked[i] = True
        else:
            end = t + hold[i] + setup
            heapq.heappush(busy, end)
            reserved += (end - t) if T_end is None else max(0.0, min(end, T_end) -
                t)
    return blocked, reserved
def dataset(kind, seed, T=7200.0, A=65.0):
    rng = np.random.default_rng(seed)
    if kind in ("voice", "data", "mix"):
        lam = A / 120.0
        n = rng.poisson(lam * T)
        arr = np.sort(rng.uniform(0, T, n))
        hold = rng.exponential(120.0, n)
        if kind == "voice":
            duty = np.ones(n)
        elif kind == "data":
            duty = np.full(n, 0.10)
        else:
            duty = np.where(rng.random(n) < 0.7, 1.0, 0.10)
        return arr, hold, duty, T
    if kind == "sync":  # 300 telemetry devices, each makes a 2 s call at the
        start of every minute
        T = 3600.0
        arr = np.repeat(np.arange(0, T, 60.0), 300) + rng.uniform(0, 0.01, 60 *
            300)  # 10 ms jitter only
        arr = np.sort(arr)
        hold = np.full(len(arr), 2.0)
        return arr, hold, np.full(len(arr), 1.0), T
    if kind == "stag":  # same calls, start times spread uniformly across the
        minute
        T = 3600.0
        base = np.repeat(np.arange(0, T, 60.0), 300)
        arr = np.sort(base + rng.uniform(0, 60.0, len(base)))
        hold = np.full(len(arr), 2.0)
        return arr, hold, np.full(len(arr), 1.0), T
def run_q2():
    rows, res = [], {}
    for kind in ("voice", "data", "mix", "sync", "stag"):
        for scheme in ("FDMA", "TDMA"):
            c, eff = CAP[scheme]["traffic"], CAP[scheme]["eff"]
            blk, util, wasted_idle, wasted_ovh, resv_frac, rep_blk = [], [], [],
                [], [], []
            for rep in range(10):
                arr, hold, duty, T = dataset(kind, seed=200 + rep)
```

```
            b, reserved = sim_loss(arr, c, hold, T_end=T)
            ok = ~b
            payload_secs = float(((hold * duty)[ok]).sum())  # seconds of
               useful transmission (full-rate equivalent)
            carried = payload_secs * eff
            cap_nominal = 80 * T
            util.append(carried / cap_nominal)
            resv_frac.append(reserved / cap_nominal)
            wasted_idle.append((float(((hold * (1 - duty))[ok]).sum())) /
               cap_nominal)
            wasted_ovh.append((payload_secs * (1 - eff) + ok.sum() * SETUP) /
               cap_nominal)
            blk.append(b.mean())
         m = lambda x: float(np.mean(x))
         sd = lambda x: float(np.std(x, ddof=1))
         offered_E = m([dataset(kind, 200 + r)[1].sum() / dataset(kind, 200 +
            r)[3] for r in range(3)])
         guard_ctrl = (80 - c) / 80
         res[f"{kind}_{scheme}"] = dict(block=m(blk), block_sd=sd(blk),
            util=m(util), util_sd=sd(util), reserved=m(resv_frac),
                                 idle=m(wasted_idle), ovh=m(wasted_ovh),
                                    gc=guard_ctrl, c=c, eff=eff)
         rows.append([kind, scheme, c, round(eff, 4), round(m(blk), 5),
            round(sd(blk), 5), round(m(util), 5), round(m(resv_frac), 5),
                  round(m(wasted_idle), 5), round(m(wasted_ovh), 5),
                     round(guard_ctrl, 5)])
   wcsv("q2_results.csv", ["dataset", "scheme", "traffic_channels",
      "payload_efficiency", "blocking", "blocking_sd", "utilisation_of_nominal",
       "reserved_fraction", "wasted_idle_fraction", "wasted_overhead_fraction",
      "guard_control_fraction"], rows)
   cap_rows = []
   for s in ("FDMA", "TDMA"):
      d = CAP[s]
      cap_rows.append([s, d["nominal"], d["guard"], d["control"], d["traffic"],
         round(d["eff"], 4), round(d["traffic"] * d["eff"], 2)])
   wcsv("q2_capacity.csv", ["scheme", "nominal", "guard_equiv", "control",
      "traffic_channels", "payload_efficiency", "payload_channel_equivalents"],
      cap_rows)
   res["telemetry"] = run_telemetry()
   return res
def run_telemetry():
   rate = 9600.0
   airtime = 200 * 8 / rate                  # 0.1667 s
   out = {}
   out["airtime"] = airtime
   out["per_report_channel_time"] = airtime + SETUP
   out["dedicated_duty_per_device"] = airtime / 60.0
   out["dedicated_devices_per_channel_ideal"] = 60.0 / (airtime + SETUP)
   rng = np.random.default_rng(31)
   rows = []
```

```
    for ndev in (40, 80, 160, 240, 320, 400, 480):
        succ = tot = 0
        sims = 40
        for s in range(sims):
            starts = []
            owner = []
            pend = [(rng.uniform(0, 60), d, 0) for d in range(ndev)]
            done = np.zeros(ndev, bool)
            events = sorted(pend)
            attempts = []
            cur = [(t, d, 0) for (t, d, _) in events]
            while cur:
                cur.sort()
                ts = np.array([x[0] for x in cur])
                collided = np.zeros(len(cur), bool)
                for i in range(len(cur)):
                    j = i + 1
                    while j < len(cur) and ts[j] < ts[i] + airtime:
                        collided[i] = collided[j] = True
                        j += 1
                nxt = []
                for k, (t, d, a) in enumerate(cur):
                    if not collided[k]:
                        done[d] = True
                    elif a < 3:
                        nxt.append((t + airtime + rng.uniform(1, 5), d, a + 1))
                cur = nxt
            succ += int(done.sum())
            tot += ndev
        rows.append([ndev, succ / tot])
    out["aloha"] = rows
    wcsv("q2_telemetry_aloha.csv", ["devices_per_channel",
        "delivery_ratio_after_3_retries"], rows)
    return out
if __name__ == "__main__":
    r1 = run_q1()
    r2 = run_q2()
    json.dump(dict(q1=r1, q2=r2), open(OUT + "q1q2.json", "w"), indent=1,
        default=float)
    print("ok")
```

File: sim_q3.py

```
"""Q3 DS-CDMA uplink simulation (BPSK, direct sequence, AWGN). Fixed seeds; CSV
in
    out/."""
import json, csv, math
import numpy as np
from scipy.linalg import hadamard
from scipy.special import erfc
OUT = "/home/claude/w2/out/"
```

```
EBN0_DB = 10.0
NBITS = 100000
CHUNK = 20000
def Q(x):
    return 0.5 * erfc(x / math.sqrt(2))
def walsh(L):
    return hadamard(L).astype(float)  # rows are Walsh codes (Hadamard order),
        +/-1
def rand_codes(K, L, rng):
    return rng.choice([-1.0, 1.0], size=(K, L))
def noise_sigma(L, ebn0_db):
    """chip amplitude a = 1 -> Eb = L; noise variance per chip sigma^2 = N0/2 =
        Eb/(2 Eb/N0) = L/(2 EbN0)."""
    return math.sqrt(L / (2 * 10 ** (ebn0_db / 10)))
BLK = 1000
def simulate(codes_fn, amps, sync=True, nbits=NBITS, seed=0, ebn0_db=EBN0_DB):
    """User 0 is the desired user. codes_fn(rng) returns a K x L code matrix; it
        is called once per block of
    BLK bits so that random codes and random chip delays are averaged over many
        draws.
    Returns bit errors of user 0 and the mean |normalised cross-correlation|
        between user 0 and the others."""
    rng = np.random.default_rng(seed)
    errs = 0
    done = 0
    corr = []
    while done < nbits:
        nb = min(BLK, nbits - done)
        codes = codes_fn(rng)
        K, L = codes.shape
        sig = noise_sigma(L, ebn0_db)
        bits = rng.choice([-1.0, 1.0], size=(nb + 1, K))
        if sync:
            r = (bits[:nb] * amps[None, :]) @ codes + rng.normal(0, sig, size=(nb,
                L))
        else:
            r = (bits[:nb, 0] * amps[0])[:, None] * codes[0][None, :]
            for k in range(1, K):
                d = int(rng.integers(1, L))          # integer chip delay of
                    interferer k (1 .. L-1 chips)
                stream = (bits[:, k, None] * codes[k][None, :]).reshape(-1) *
                    amps[k]
                r = r + stream[(L - d):(L - d) + nb * L].reshape(nb, L)
            r = r + rng.normal(0, sig, size=(nb, L))
        z = r @ codes[0]                             # despread: correlate
            with own code, sum over L chips
        dec = np.where(z >= 0, 1.0, -1.0)                # hard decision
        errs += int((dec != bits[:nb, 0]).sum())
        if K > 1:
            corr.append(float((np.abs(codes[1:] @ codes[0]) / L).mean()))
```

```
        done += nb
    return errs, (float(np.mean(corr)) if corr else 0.0)
def walsh_fn(L, K):
    W = walsh(L)
    C = W[1:K + 1] if K < L else W[:K]
    return lambda rng: C
def rand_fn(L, K):
    return lambda rng: rand_codes(K, L, rng)
def mean_abs_corr(codes):
    K, L = codes.shape
    if K < 2:
        return 0.0
    G = np.abs(codes @ codes.T) / L
    return float(G[np.triu_indices(K, 1)].mean())
def theory_ber(K, L, ebn0_db, power_ratio=1.0, kind="random"):
    """Standard Gaussian approximation: random codes: SINR^-1 = (sum_k P_k/P_0)/L
        + N0/(2Eb);
    orthogonal synchronous codes: MAI = 0."""
    n0 = 1.0 / (2 * 10 ** (ebn0_db / 10))
    if kind == "walsh":
        mai = 0.0
    else:
        mai = ((K - 1) * power_ratio) / L
    return Q(1.0 / math.sqrt(mai + n0))
def theory_nf(K, L, ebn0_db, power_ratio):
    """Random codes, one user at power_ratio times the others: MAI = (K - 2 +
        power_ratio)/L."""
    n0 = 1.0 / (2 * 10 ** (ebn0_db / 10))
    return Q(1.0 / math.sqrt((max(K - 2, 0) + power_ratio) / L + n0))
def run():
    rows = []
    res = {}
    Ls = {8: [1, 2, 4, 6, 8], 16: [1, 2, 4, 8, 12, 16], 64: [1, 4, 8, 16, 32, 48,
        64]}
    for L, Ks in Ls.items():
        for K in Ks:
            amps = np.ones(K)
            a3 = np.ones(K)
            if K >= 2:
                a3[1] = math.sqrt(10.0)
            e1, c1 = simulate(walsh_fn(L, K), amps, True, seed=1000 + L + K)
                        # (i) Walsh, sync, equal power
            e2, c2 = simulate(rand_fn(L, K), amps, True, seed=3000 + L + K)
                          # (ii) random, sync, equal power
            e3, _ = simulate(walsh_fn(L, K), a3, True, seed=4000 + L + K)
                           # (iii) Walsh, sync, one user 10x power
            e4, c4 = simulate(walsh_fn(L, K), amps, False, seed=5000 + L + K)
                        # (iv) Walsh, asynchronous
            e5, _ = simulate(rand_fn(L, K), a3, True, seed=6000 + L + K)
                           # (v) random, one user 10x power
```

```
        e6, _ = simulate(walsh_fn(L, K), a3, False, seed=7000 + L + K)
                    # (vi) Walsh, async, one user 10x power
        rows.append([L, K, NBITS, e1, e2, e3, e4, e5, e6, round(c1, 4),
           round(c2, 4), round(c4, 4), round(10 * math.log10(L), 2)])
        res[f"{L}_{K}"] = dict(L=L, K=K, walsh=e1, rand=e2, nearfar=e3,
           asyn=e4, rand_nf=e5, asyn_nf=e6, corr_walsh=c1, corr_rand=c2,
                       corr_asyn=c4, pg_db=10 * math.log10(L),
                       th_walsh=theory_ber(K, L, EBN0_DB,
                          kind="walsh"), th_rand=theory_ber(K, L,
                          EBN0_DB),
                       th_rand_nf=theory_nf(K, L, EBN0_DB, 10.0) if K
                          >= 2 else 0.0)
with open(OUT + "q3_ber.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["L", "K", "bits_user0", "err_walsh_sync", "err_random_sync",
       "err_walsh_sync_nearfar10x", "err_walsh_async",
       "err_random_sync_nearfar10x", "err_walsh_async_nearfar10x",
       "mean_abs_xcorr_walsh_sync", "mean_abs_xcorr_random",
       "mean_abs_xcorr_walsh_async", "processing_gain_dB"])
    w.writerows(rows)
cap = {}
for L in (8, 16, 64):
    bers = {"random": [], "walsh_async": [], "walsh_sync": []}
    nb = 40000
    for K in range(1, L + 1):
        amps = np.ones(K)
        bers["random"].append(simulate(rand_fn(L, K), amps, True, nbits=nb,
           seed=8000 + L + K)[0] / nb)
        bers["walsh_async"].append(simulate(walsh_fn(L, K), amps, False,
           nbits=nb, seed=9000 + L + K)[0] / nb)
        bers["walsh_sync"].append(simulate(walsh_fn(L, K), amps, True,
           nbits=nb, seed=9500 + L + K)[0] / nb)
    kmax = {}
    for key, arr in bers.items():
        k = 0
        for v in arr:
            if v <= 1e-2:
                k += 1
            else:
                break
        kmax[key] = k
    cap[L] = dict(kmax=kmax, bers=bers)
with open(OUT + "q3_capacity.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["L", "K", "BER_random_sync", "BER_walsh_async",
       "BER_walsh_sync"])
    for L in cap:
        for K in range(1, L + 1):
            w.writerow([L, K, cap[L]["bers"]["random"][K - 1],
               cap[L]["bers"]["walsh_async"][K - 1],
```

```
                cap[L]["bers"]["walsh_sync"][K - 1]])
    nf = []
    L, K = 64, 8
    for pr_db in (0, 3, 6, 10, 13, 16, 20, 25, 30):
        a = np.ones(K)
        a[1] = 10 ** (pr_db / 20)
        nb = 60000
        b1 = simulate(walsh_fn(L, K), a, True, nbits=nb, seed=11000 + pr_db)[0] /
            nb
        b2 = simulate(walsh_fn(L, K), a, False, nbits=nb, seed=12000 + pr_db)[0] /
             nb
        b3 = simulate(rand_fn(L, K), a, True, nbits=nb, seed=13000 + pr_db)[0] /
            nb
        nf.append([pr_db, b1, b2, b3, theory_nf(K, L, EBN0_DB, 10 ** (pr_db /
            10))])
    with open(OUT + "q3_nearfar.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["interferer_power_dB", "BER_walsh_sync", "BER_walsh_async",
            "BER_random_sync", "BER_random_theory_gaussian"])
        w.writerows(nf)
    res["cap"] = {str(k): dict(kmax=v["kmax"], bers=v["bers"]) for k, v in
        cap.items()}
    res["nf"] = nf
    W64 = walsh(64)
    G = W64 @ W64.T
    res["walsh_orth_max_offdiag"] = float(np.abs(G - 64 * np.eye(64)).max())
    shifts = []
    for a_ in range(1, 8):
        for b_ in range(a_ + 1, 8):
            for s_ in range(1, 64):
                shifts.append(float(np.abs(W64[a_] @ np.roll(W64[b_], s_)) / 64))
    res["walsh_shift_xcorr_mean"] = float(np.mean(shifts))
    res["walsh_shift_xcorr_max"] = float(np.max(shifts))
    json.dump(res, open(OUT + "q3.json", "w"), indent=1, default=float)
    return res
if __name__ == "__main__":
    import time
    t = time.time()
    run()
    print("done", time.time() - t)
```

File: sim_q4q5.py

```
"""Q4 OFDMA allocation and Q5 SDMA uniform-linear-array simulations. Fixed seeds;
    CSVs in out/."""
import json, csv, math
import numpy as np
OUT = "/home/claude/w2/out/"
def wcsv(name, header, rows):
    with open(OUT + name, "w", newline="") as f:
        w = csv.writer(f)
```

```
        w.writerow(header)
        w.writerows(rows)
NSC, NU, NREAL = 64, 8, 2000
SPACING, TU, TCP = 312.5e3, 3.2e-6, 0.8e-6      # IEEE 802.11a-style numerology
    (20 MHz / 64)
SYM_RATE = 1 / (TU + TCP)                       # 250,000 OFDM symbols per second
AVG_SNR_DB = np.array([30, 27, 24, 21, 18, 15, 12, 9], float)   # users at
    increasing distance in the hall
RMIN = 8.0                                      # bits per OFDM symbol (= 2 Mb/s)
    - one video stream
def orthogonality_check():
    n = np.arange(NSC)
    ks = [3, 4, 10]
    M = np.zeros((3, 3), complex)
    for a, ka in enumerate(ks):
        for b, kb in enumerate(ks):
            M[a, b] = np.vdot(np.exp(2j * np.pi * ka * n / NSC), np.exp(2j * np.pi
                * kb * n / NSC)) / NSC
    x = np.zeros(NSC, complex)
    x[10] = 1
    tx = np.fft.ifft(x) * NSC
    delay = 5
    cp = 16
    withcp = np.concatenate([tx[-cp:], tx])
    rx = np.concatenate([np.zeros(delay), withcp])[cp:cp + NSC]         # remove
        CP after the delay
    leak_cp = np.abs(np.fft.fft(rx) / NSC)
    nocp = np.concatenate([np.zeros(delay), tx, tx * 0])[:NSC]          # symbol
        alone, delayed, no CP: truncated tail
    leak_no = np.abs(np.fft.fft(nocp) / NSC)
    leak_cp_other = float(np.delete(leak_cp, 10).max())
    leak_no_other = float(np.delete(leak_no, 10).max())
    return M, leak_cp_other, leak_no_other
def gains(rng, nreal, nusers, avg_db):
    """Frequency-selective Rayleigh channel: 6-tap exponential power-delay
        profile, FFT -> |H_k|^2 (unit mean)."""
    taps = 6
    pdp = np.exp(-np.arange(taps) / 2.0)
    pdp /= pdp.sum()
    h = (rng.normal(size=(nreal, nusers, taps)) + 1j * rng.normal(size=(nreal,
        nusers, taps))) * np.sqrt(pdp / 2)
    H = np.fft.fft(h, NSC, axis=2)
    g = np.abs(H) ** 2                                  # nreal x users x NSC
    snr = g * (10 ** (np.asarray(avg_db) / 10))[None, :, None]
    return np.log2(1 + snr)                             # bits per subcarrier
         per OFDM symbol
def alloc_block(R):
    nu = R.shape[1]
    owner = np.repeat(np.arange(nu), NSC // nu)
    return owner[None, :].repeat(R.shape[0], 0)
```

```
def alloc_interleaved(R):
    nu = R.shape[1]
    owner = np.arange(NSC) % nu
    return owner[None, :].repeat(R.shape[0], 0)
def alloc_best(R):
    return R.argmax(axis=1)                              # nreal x NSC: user
        with the best rate on each subcarrier
def alloc_constrained(R, rmin):
    """Channel-aware allocation with a minimum-rate guarantee: first give every
        user, in order of weakest average first,
    its best remaining subcarriers until its rate reaches rmin (greedy), then give
         the remaining subcarriers to the best user."""
    nreal, nu, nsc = R.shape
    owner = -np.ones((nreal, nsc), int)
    for r in range(nreal):
        free = np.ones(nsc, bool)
        rate = np.zeros(nu)
        order = np.argsort(R[r].mean(axis=1))
        for u in order:
            idx = np.argsort(-R[r, u])
            for k in idx:
                if rate[u] >= rmin:
                    break
                if free[k]:
                    owner[r, k] = u
                    free[k] = False
                    rate[u] += R[r, u, k]
        rest = np.where(free)[0]
        for k in rest:
            owner[r, k] = R[r, :, k].argmax()
    return owner
def metrics(R, owner):
    nreal, nu, nsc = R.shape
    per = np.zeros((nreal, nu))
    for u in range(nu):
        per[:, u] = (R[:, u, :] * (owner == u)).sum(axis=1)
    tot = per.sum(axis=1)
    jain = (per.sum(axis=1) ** 2) / (nu * (per ** 2).sum(axis=1) + 1e-12)
    return per, tot, jain
def run_q4():
    res = {}
    M, leak_cp, leak_no = orthogonality_check()
    res["orth_matrix"] = np.abs(M).tolist()
    res["leak_cp"], res["leak_nocp"] = leak_cp, leak_no
    rng = np.random.default_rng(404)
    R = gains(rng, NREAL, NU, AVG_SNR_DB)
    rows = []
    for name, fn in (("fixed contiguous blocks", alloc_block), ("interleaved
        round-robin", alloc_interleaved), ("channel-aware best subcarrier",
        alloc_best)):
```

```
        owner = fn(R)
        per, tot, jain = metrics(R, owner)
        out_p = (per < RMIN).mean(axis=0)
        res[name] = dict(sum=float(tot.mean()), sum_sd=float(tot.std()),
            per=per.mean(axis=0).tolist(), per_min=per.min(axis=0).tolist(),
            jain=float(jain.mean()), outage=out_p.tolist(), outage_any=float((per
            < RMIN).any(axis=1).mean()))
        rows.append([name, round(tot.mean(), 2), round(jain.mean(), 4)] +
            [round(x, 2) for x in per.mean(axis=0)] + [round(float((per <
            RMIN).any(axis=1).mean()), 4)])
    owner = alloc_constrained(R, RMIN)
    per, tot, jain = metrics(R, owner)
    name = "channel-aware with minimum-rate guarantee"
    res[name] = dict(sum=float(tot.mean()), sum_sd=float(tot.std()),
        per=per.mean(axis=0).tolist(), per_min=per.min(axis=0).tolist(),
        jain=float(jain.mean()), outage=(per < RMIN).mean(axis=0).tolist(),
        outage_any=float((per < RMIN).any(axis=1).mean()))
    rows.append([name, round(tot.mean(), 2), round(jain.mean(), 4)] + [round(x, 2)
         for x in per.mean(axis=0)] + [round(float((per <
        RMIN).any(axis=1).mean()), 4)])
    wcsv("q4_allocation.csv", ["scheme", "sum_bits_per_symbol", "jain"] +
        [f"user{u + 1}_bits_per_symbol" for u in range(NU)] + ["P(any user below
        Rmin)"], rows)
    div = []
    for nu in (1, 2, 4, 8, 16, 32):
        rng = np.random.default_rng(500 + nu)
        Ru = gains(rng, 1000, nu, np.full(nu, 15.0))
        if nu <= NSC:
            if NSC % nu == 0:
                ownerI = alloc_interleaved(Ru)
            else:
                ownerI = (np.arange(NSC) % nu)[None, :].repeat(1000, 0)
        _, tI, jI = metrics(Ru, ownerI)
        _, tB, jB = metrics(Ru, alloc_best(Ru))
        div.append([nu, float(tI.mean()), float(tB.mean()), float(tB.mean() /
            tI.mean()), float(jB.mean())])
    wcsv("q4_multiuser_diversity.csv", ["users", "sum_interleaved", "sum_best",
        "gain_ratio", "jain_best"], div)
    res["div"] = div
    res["numerology"] = dict(sym_rate=SYM_RATE, spacing=SPACING, tu=TU,
tcp=TCP)
    res["cp_overhead"] = TCP / (TU + TCP)
    return res
D_OVER_LAMBDA = 0.5
def af(theta_deg, theta0_deg, N, d=D_OVER_LAMBDA):
    """Normalised array factor of an N-element uniform linear array steered to
        theta0 (angles from broadside)."""
    psi = 2 * np.pi * d * (np.sin(np.radians(theta_deg)) -
        np.sin(np.radians(theta0_deg)))
    num = np.sin(N * psi / 2)
```

```
    den = N * np.sin(psi / 2)
    out = np.where(np.abs(den) < 1e-12, 1.0, num / np.where(np.abs(den) < 1e-12,
        1.0, den))
    return out
def pattern_stats(N, theta0=0.0):
    """Half-power beamwidth, highest side lobe (dB) and null-to-null width for a
        ULA steered to theta0."""
    th = np.linspace(-90, 90, 36001)
    pdb = 10 * np.log10(np.abs(af(th, theta0, N)) ** 2 + 1e-12)
    idx0 = int(np.argmin(np.abs(th - theta0)))
    lo = idx0
    while lo > 0 and pdb[lo] >= -3.0103:
        lo -= 1
    hi = idx0
    while hi < len(th) - 1 and pdb[hi] >= -3.0103:
        hi += 1
    hpbw = th[hi] - th[lo]
    s0 = np.sin(np.radians(theta0))
    nulls = [np.degrees(np.arcsin(s0 + sgn / (N * D_OVER_LAMBDA))) for sgn in (-1,
         1) if abs(s0 + sgn / (N * D_OVER_LAMBDA)) < 1]
    lo_n, hi_n = (min(nulls), max(nulls)) if len(nulls) == 2 else (-90, 90)
    side = [pdb[i] for i in range(1, len(th) - 1) if pdb[i] > pdb[i - 1] and
        pdb[i] >= pdb[i + 1] and (th[i] < lo_n - 1e-6 or th[i] > hi_n + 1e-6)]
    return hpbw, (max(side) if side else -99.0), hi_n - lo_n
def run_q5():
    res = {}
    stats = []
    for N in (2, 4, 8, 16):
        h, s, bn = pattern_stats(N, 0.0)
        h40, _, bn40 = pattern_stats(N, 40.0)
        stats.append([N, h, s, bn, h40, 50.76 / N, 2 * math.degrees(math.asin(2.0
            / N)) if N > 2 else 180.0])
    res["stats"] = stats
    wcsv("q5_pattern_stats.csv", ["N", "HPBW_deg_broadside", "first_sidelobe_dB",
        "BWNN_deg", "HPBW_deg_at_40deg_scan", "HPBW_approx_50.76/N",
        "BWNN_analytic_deg"], stats)
    seps = np.arange(5, 61, 1.0)
    sir = {}
    rows = []
    for N in (2, 4, 8, 16):
        for theta_a in (0.0, 40.0):
            s_db = []
            for sep in seps:
                tb = theta_a + sep
                if tb > 90:
                    tb = theta_a - sep
                gB = np.abs(af(np.array([tb]), theta_a, N)[0]) ** 2
                s_db.append(-10 * np.log10(gB + 1e-12))              # SIR = P_A
                    * 1 / (P_B * |AF(B)|^2)
            sir[f"{N}_{theta_a}"] = s_db
```

```
for i, sep in enumerate(seps):
    rows.append([sep] + [round(sir[f"{N}_0.0"][i], 2) for N in (2, 4, 8, 16)]
        + [round(sir[f"{N}_40.0"][i], 2) for N in (2, 4, 8, 16)])
wcsv("q5_sir_vs_separation.csv", ["separation_deg"] + [f"SIR_dB_N{N}_scan0"
    for N in (2, 4, 8, 16)] + [f"SIR_dB_N{N}_scan40" for N in (2, 4, 8, 16)],
    rows)
res["seps"] = seps.tolist()
res["sir"] = sir
thr = 12.0
minsep = {}
for key, s_db in sir.items():
    ok = np.array(s_db) >= thr
    m = None
    for i in range(len(seps)):
        if ok[i:].all():
            m = float(seps[i])
            break
    first = None
    for i in range(len(seps)):
        if ok[i]:
            first = float(seps[i])
            break
    minsep[key] = (first, m)
res["minsep"] = minsep
res["thr"] = thr
rng = np.random.default_rng(55)
spread = []
for sigma in (0.0, 1.0, 2.0, 4.0):
    row = [sigma]
    for N in (4, 8, 16):
        a = rng.normal(0, sigma, 4000)
        b = 20.0 + rng.normal(0, sigma, 4000)
        ga = np.abs(af(a, 0.0, N)) ** 2
        gb = np.abs(af(b, 0.0, N)) ** 2
        sir_r = 10 * np.log10(ga / (gb + 1e-12) + 1e-12)
        row.append(float((sir_r >= thr).mean()))
    spread.append(row)
res["spread"] = spread
wcsv("q5_angular_spread.csv", ["sigma_deg", "P(SIR>=12dB) N=4 sep=20deg",
    "N=8", "N=16"], spread)
rng = np.random.default_rng(66)
toll = []
for w in (5.0, 10.0, 30.0, 60.0):
    a = rng.uniform(-w, w, 100000)
    b = rng.uniform(-w, w, 100000)
    d = np.abs(a - b)
    row = [w]
    for N in (4, 8, 16):
        ms = minsep[f"{N}_0.0"][1]
        ms = ms if ms is not None else 60.0
```

```
            row.append(float((d >= ms).mean()))
        toll.append(row)
    res["toll"] = toll
    wcsv("q5_toll_plaza.csv", ["half_width_deg", "P(pair separable) N=4", "N=8",
        "N=16"], toll)
    return res
if __name__ == "__main__":
    r4 = run_q4()
    r5 = run_q5()
    json.dump(dict(q4=r4, q5=r5), open(OUT + "q4q5.json", "w"), indent=1,
        default=float)
    print("ok")
```

File: sim_q6q7.py

```
"""Q6 weighted decision matrix (+/-20% weight sensitivity) and Q7 redesign:
    dedicated channel per session vs
burst-level reservation with hangover timer and contention mini-slots. Fixed
    seeds; CSVs in out/."""
import json, csv, itertools
import numpy as np
OUT = "/home/claude/w2/out/"
def wcsv(n, h, rows):
    with open(OUT + n, "w", newline="") as f:
        w = csv.writer(f); w.writerow(h); w.writerows(rows)
TECH = ["SDMA", "TDMA", "FDMA", "CDMA", "OFDMA"]
CRIT = ["spectral efficiency", "bursty-traffic efficiency", "interference/jam
    robustness", "implementation simplicity", "power-control / sync burden (higher
     = lighter)"]
DEPL = ["wide-area voice", "tactical jam-resistant", "stadium downlink", "5,000
    sensors"]
W = {  # weights per deployment (sum 1), justified in the report
 "wide-area voice":        [0.30, 0.15, 0.15, 0.20, 0.20],
 "tactical jam-resistant": [0.10, 0.10, 0.50, 0.10, 0.20],
 "stadium downlink":       [0.35, 0.20, 0.10, 0.10, 0.25],
 "5,000 sensors":          [0.10, 0.40, 0.05, 0.25, 0.20]}
S = {
 "wide-area voice":
    [[3,2,3,2,3],[4,3,3,3,4],[3,2,3,4,5],[4,4,4,2,2],[4,4,3,2,3]],
 "tactical jam-resistant":
    [[3,2,3,2,3],[2,3,2,3,4],[1,1,1,4,5],[5,4,5,2,2],[2,3,2,2,3]],
 "stadium downlink":
    [[4,3,3,1,2],[3,3,2,3,4],[2,1,2,4,5],[3,3,3,2,2],[5,5,4,3,3]],
 "5,000 sensors":
    [[1,2,2,1,2],[2,2,2,3,3],[1,1,2,4,4],[3,4,4,2,2],[3,4,3,3,3]]}
def best(dep, w):
    sc = np.array(S[dep]) @ np.array(w)
    return TECH[int(sc.argmax())], sc
def q6():
    res, rows, sens = {}, [], []
    for d in DEPL:
```

```
        b, sc = best(d, W[d]); res[d] = dict(best=b, scores=sc.tolist())
        rows.append([d] + [round(x, 3) for x in sc] + [b])
        flips, tot = 0, 0
        for i in range(5):
            for f in (0.8, 1.2):
                w = np.array(W[d]); w[i] *= f; w /= w.sum()
                bb, _ = best(d, w); tot += 1
                if bb != b: flips += 1; sens.append([d, CRIT[i], f, b, bb])
        res[d]["flips"] = f"{flips}/{tot}"
        srt = np.sort(sc)[::-1]; res[d]["margin"] = float(srt[0] - srt[1])
    wcsv("q6_decision_matrix.csv", ["deployment"] + TECH + ["primary"], rows)
    wcsv("q6_sensitivity_flips.csv", ["deployment", "criterion", "weight_factor",
        "base_choice", "new_choice"], sens)
    res["sens"] = sens
    return res
C = 70; SLOT = 0.1; MEAN_SESSION = 3000; BURST = 10; OFF = 90      # 7 s... slots:
     burst 1 s, off 9 s -> 10 % duty
HANG = 3; NMINI = 4; PREQ = 0.3; T = 45000; WARM = 15000; MAXWAIT = 20
def run(A, mode, seed=7):
    """One run. A burst is a fixed amount of data (geometric, mean BURST slots)
        that is sent only while the terminal holds
    a channel, so a burst that has to wait is delayed, never shortened. Gaps
        between bursts (mean OFF slots) start when the
    previous burst has been fully sent."""
    rng = np.random.default_rng(seed)
    lam = A / MEAN_SESSION
    U = []
    used = 0; burstslots = 0; held_slots = 0; blocked = 0; arrived = 0
    req = 0; coll = 0; bursts = 0; dropped = 0; delaysum = 0; granted = 0; setups
        = 0
    for t in range(T):
        meas = t >= WARM
        for _ in range(rng.poisson(lam)):
            if meas: arrived += 1
            if mode == "dedicated" and used >= C:
                if meas: blocked += 1
                continue
            u = dict(end=t + rng.geometric(1 / MEAN_SESSION), on=False,
                rem=int(rng.geometric(1 / OFF)), need=0, hold=False, hang=0,
                wait=-1)
            if mode == "dedicated":
                u["hold"] = True; used += 1
                if meas: setups += 1
            U.append(u)
        for u in U:
            if u["on"]:
                if u["hold"]:
                    if meas: burstslots += 1
                    u["need"] -= 1
                    if u["need"] <= 0:
```

```
                u["on"] = False; u["wait"] = -1; u["rem"] =
                    int(rng.geometric(1 / OFF))
                if mode == "reserve": u["hang"] = HANG
        else:
            u["rem"] -= 1
            if u["rem"] <= 0:
                u["on"] = True; u["need"] = int(rng.geometric(1 / BURST));
                    u["wait"] = 0
                if meas: bursts += 1
    if mode == "reserve":
        for u in U:
            if u["hold"] and not u["on"]:
                u["hang"] -= 1
                if u["hang"] <= 0: u["hold"] = False; used -= 1
        pend = [u for u in U if u["on"] and not u["hold"]]
        for u in pend: u["wait"] += 1
        choices = {}
        for i, u in enumerate(pend):
            if rng.random() < PREQ:
                choices.setdefault(int(rng.integers(NMINI)), []).append(i)
                if meas: req += 1
        for m, lst in choices.items():
            if len(lst) == 1:
                u = pend[lst[0]]
                if used < C:
                    u["hold"] = True; used += 1
                    if meas: setups += 1; granted += 1; delaysum += u["wait"]
            elif meas: coll += 1
        for u in pend:
            if u["wait"] > MAXWAIT and not u["hold"]:
                u["on"] = False; u["wait"] = -1; u["rem"] =
                    int(rng.geometric(1 / OFF))
                if meas: dropped += 1
    if meas:
        held_slots += sum(1 for u in U if u["hold"])
    keep = []
    for u in U:
        if u["end"] <= t:
            if u["hold"]: used -= 1
        else: keep.append(u)
    U = keep
dur = T - WARM
return dict(A=A, mode=mode, arrived=arrived, blocked=blocked, block_pct=100 *
    blocked / max(arrived, 1),
            util_pct=100 * burstslots / max(held_slots, 1),
                mean_channels_held=held_slots / dur,
            wasted_pct=100 * (1 - burstslots / max(held_slots, 1)),
                served_burst_slots=burstslots,
            setup_msgs=setups, requests=req, collisions=coll, bursts=bursts,
                dropped_bursts=dropped,
```

```
            drop_pct=100 * dropped / max(bursts, 1),
               mean_access_delay_slots=delaysum / max(granted, 1))
def q7():
   rows, res = [], []
   for A in (20, 40, 60, 80, 100, 120, 160, 240):
      for mode in ("dedicated", "reserve"):
         r = run(A, mode, seed=700 + A); res.append(r)
         rows.append([r[k] for k in ("A", "mode", "arrived", "blocked",
            "block_pct", "util_pct", "mean_channels_held", "wasted_pct",
            "served_burst_slots", "setup_msgs", "requests", "collisions",
            "bursts", "dropped_bursts", "drop_pct",
            "mean_access_delay_slots")])
   wcsv("q7_original_vs_redesign.csv", ["offered_sessions_erlang", "design",
      "sessions_arrived", "sessions_blocked", "session_block_pct",
      "channel_utilisation_pct", "mean_channels_held", "wasted_pct",
      "burst_slots_served", "setup_msgs", "contention_requests", "collisions",
      "bursts", "bursts_dropped", "burst_drop_pct", "mean_access_delay_slots"],
      rows)
   return res
if __name__ == "__main__":
   json.dump(dict(q6=q6(), q7=q7()), open(OUT + "q6q7.json", "w"), indent=1,
      default=float)
   print("ok")
```

ANNEXURE B – Raw data files (CSV) supplied with this report

Table 29: CSV files

File	Content	Data rows
q1_main.csv	Q1 counts for both strategies at 90/60/30% availability	6
q1_sensitivity.csv	Q1 outage length and queue size	12
q2_capacity.csv	Q2 usable capacity	2
q2_results.csv	Q2 utilisation, blocking, waste	10
q2_telemetry_aloha.csv	Q2 telemetry random access	7
q3_ber.csv	Q3 bit errors, correlations, processing gain	18
q3_capacity.csv	Q3 BER vs K	88
q3_nearfar.csv	Q3 near-far sweep	9
q4_allocation.csv	Q4 four allocation schemes	4
q4_multiuser_diversity.csv	Q4 gain vs number of users	6
q5_pattern_stats.csv	Q5 beamwidth, side lobes	4
q5_sir_vs_separation.csv	Q5 SIR 5–60°	56
q5_angular_spread.csv	Q5 angular spread	4
q5_toll_plaza.csv	Q5 toll plaza	4
q6_decision_matrix.csv	Q6 weighted scores	4
q6_sensitivity_flips.csv	Q6 weight changes that alter the choice	1
q7_original_vs_redesign.csv	Q7 original versus redesign	16